



US 20250280110A1

(19) **United States**

(12) **Patent Application Publication**

(10) **Pub. No.: US 2025/0280110 A1**

(43) **Pub. Date: Sep. 4, 2025**

(12) **LAROCHE et al.**

(54) **HIGH LEVEL SYNTAX FOR VIDEO CODING AND DECODING**

(71) Applicant: **CANON KABUSHIKI KAISHA**,
Tokyo (JP)

(72) Inventors: **Guillaume LAROCHE**, Saint Aubin
d'Aubigné (FR); **Naël OUEDRAOGO**,
Val d'Anast (FR); **Patrice ONNO**,
RENNES (FR)

(21) Appl. No.: **19/210,711**

(22) Filed: **May 16, 2025**

Related U.S. Application Data

(63) Continuation of application No. 17/910,745, filed on
Sep. 9, 2022, filed as application No. PCT/EP2021/
055661 on Mar. 5, 2021, now Pat. No. 12,363,278.

Foreign Application Priority Data

Mar. 11, 2020 (GB) 2003562.2
Apr. 20, 2020 (GB) 2005741.0

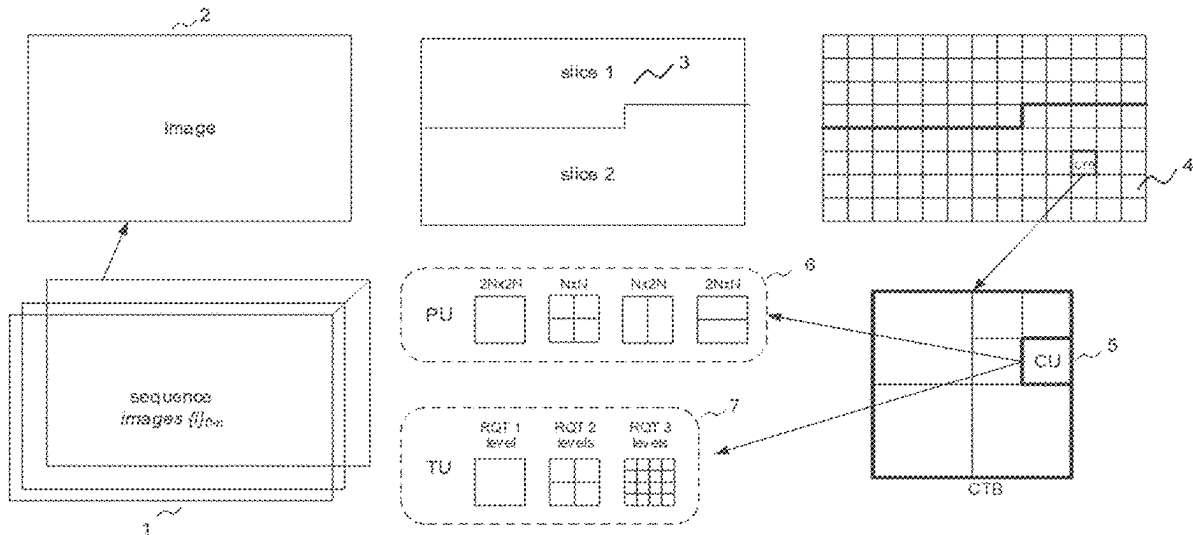
Publication Classification

(51) **Int. Cl.**
H04N 19/105 (2014.01)
H04N 19/174 (2014.01)
H04N 19/46 (2014.01)
H04N 19/70 (2014.01)

(52) **U.S. Cl.**
CPC **H04N 19/105** (2014.11); **H04N 19/174**
(2014.11); **H04N 19/46** (2014.11); **H04N**
19/70 (2014.11)

(57) **ABSTRACT**

A method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices. The bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice. The decoding comprises in a case where information which can be signalled in the picture header or in the slice header is signalled in the picture header, a picture header is to be signalled in a slice header, permitting parsing of information which may otherwise be signalled in a slice header and a picture header, in only one of the slice header or picture header; imposing that the picture header is not in the slice header, and decoding said bitstream using said syntax elements.



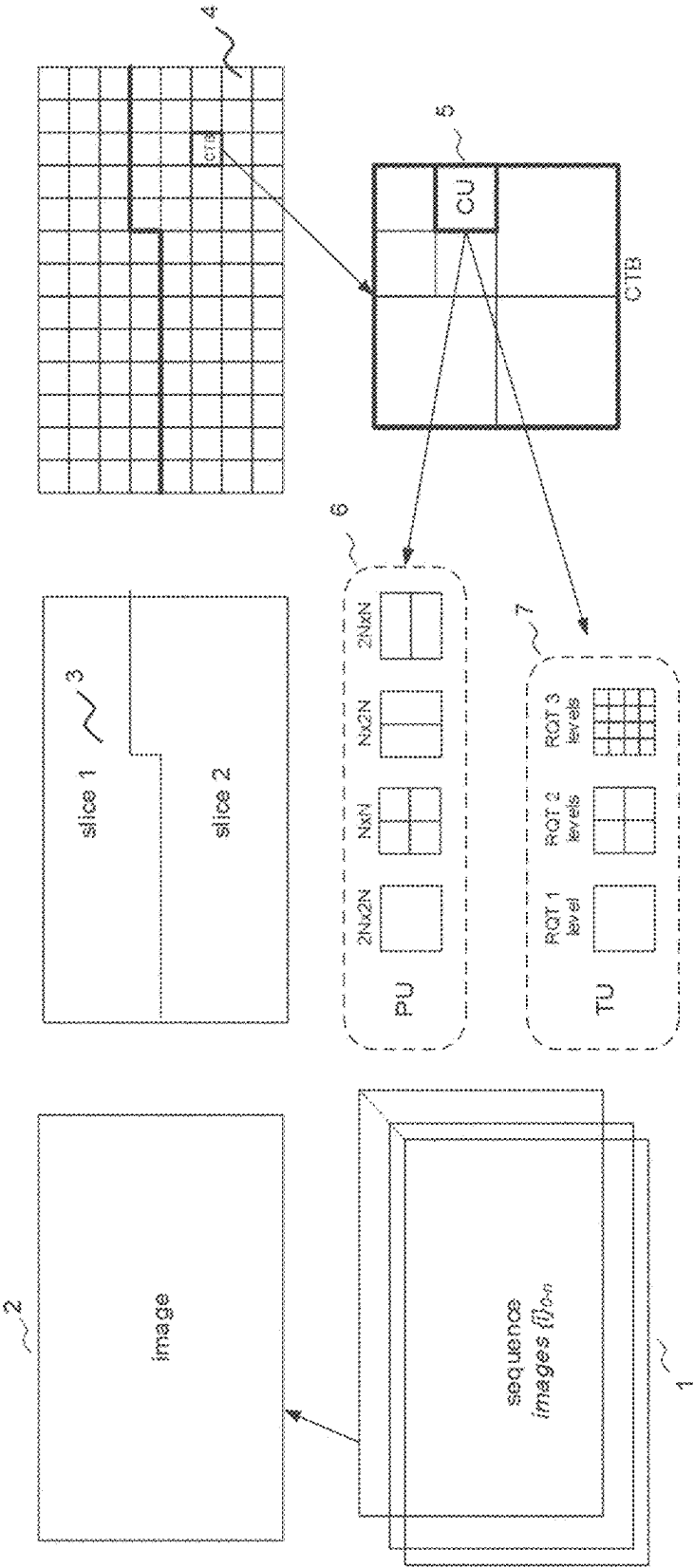


Figure 1

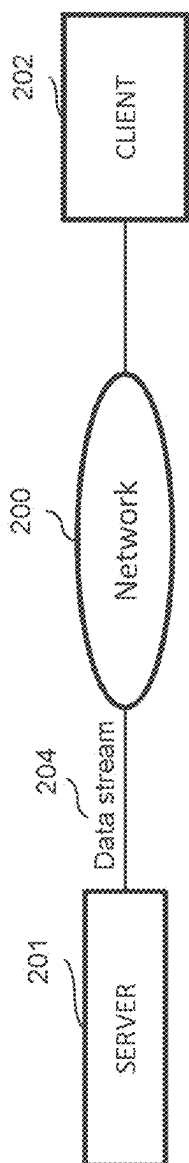


Figure 2

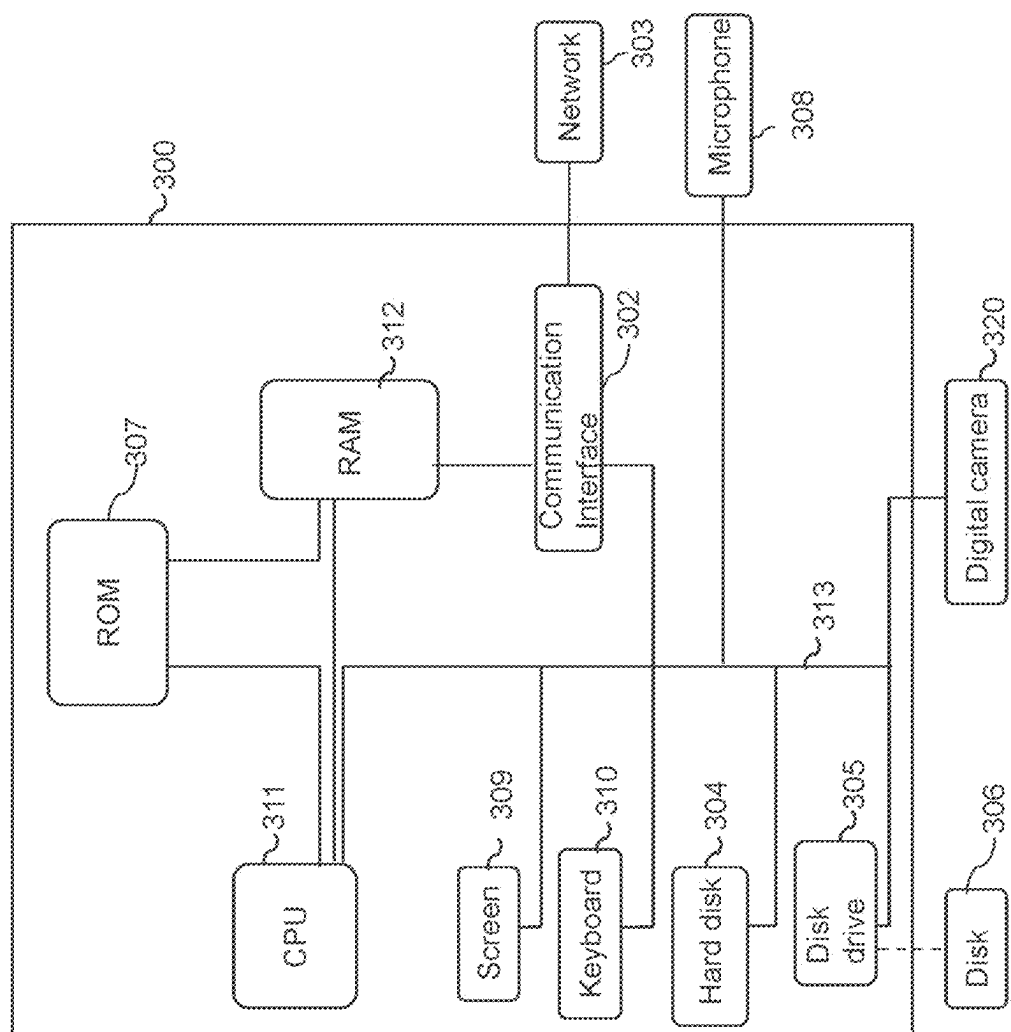


Figure 3

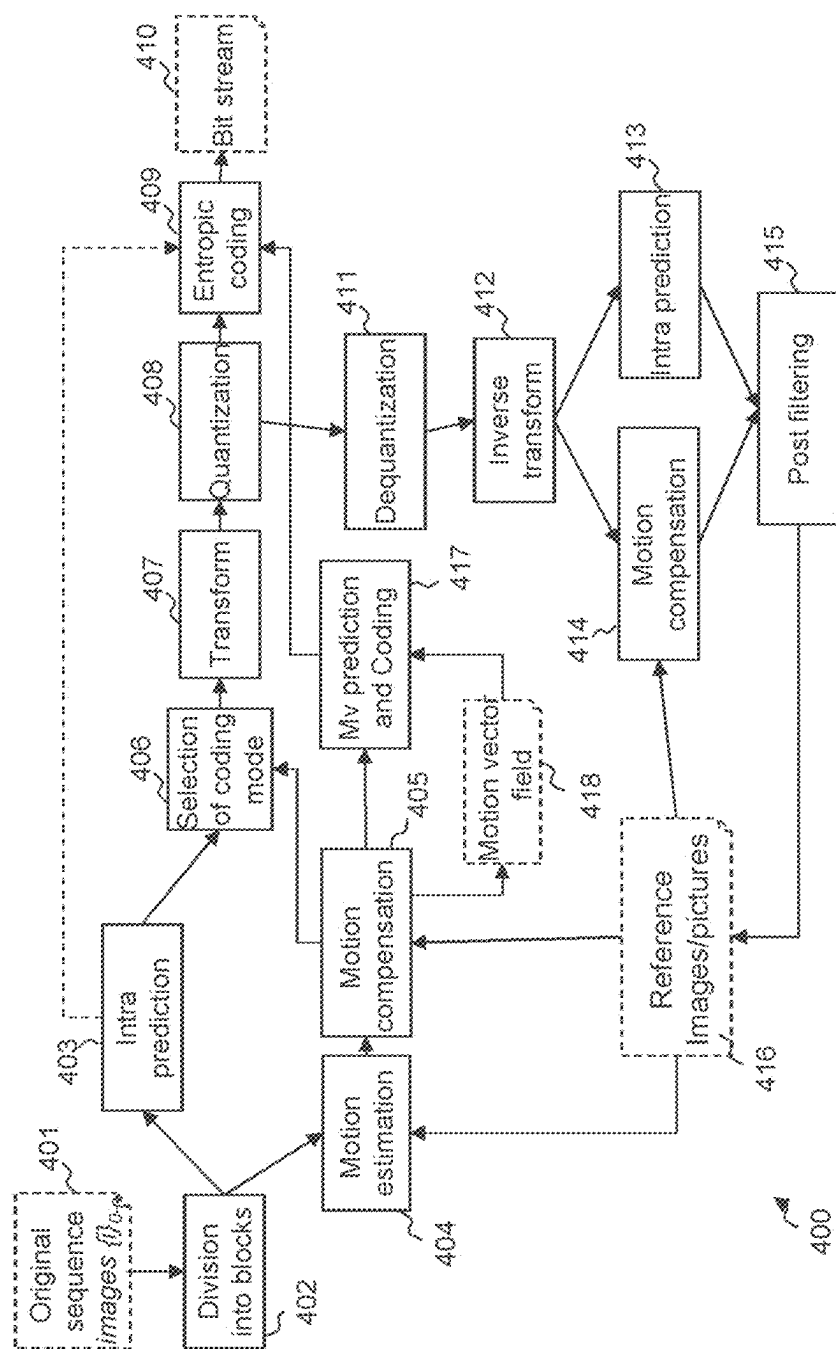


Figure 4

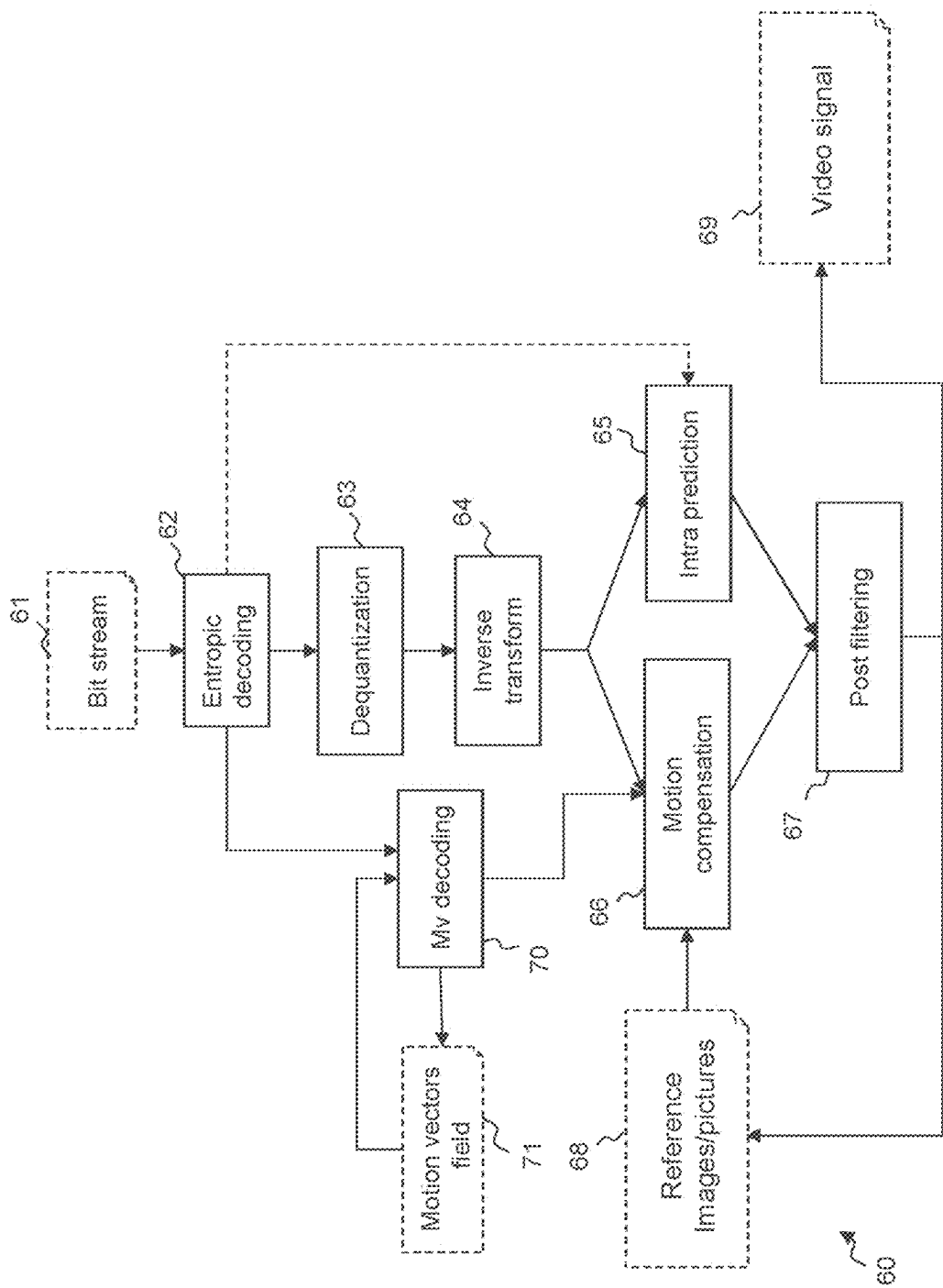


Figure 5

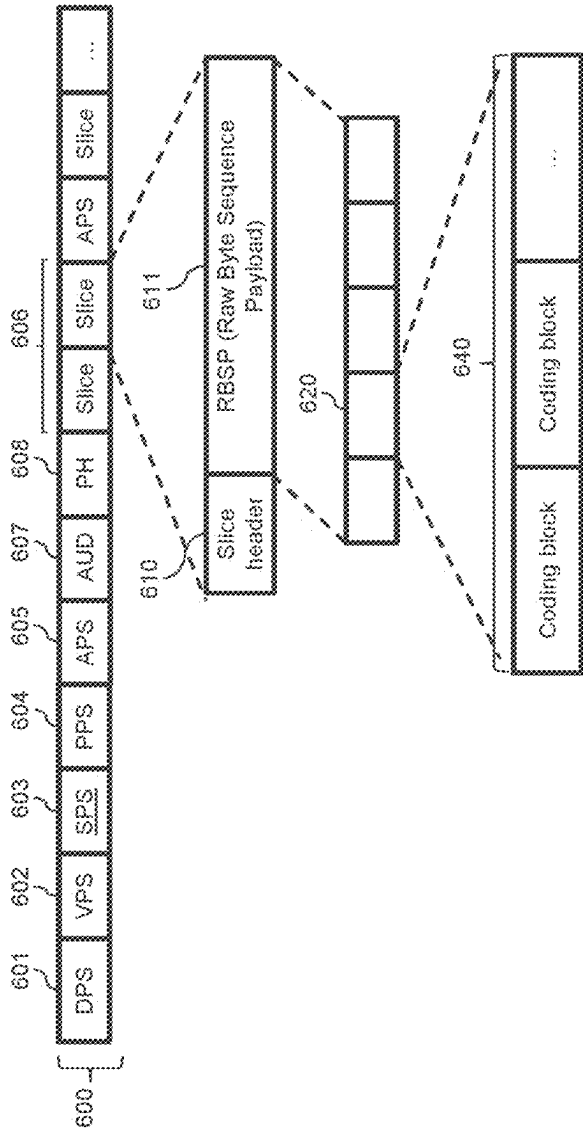


Figure 6

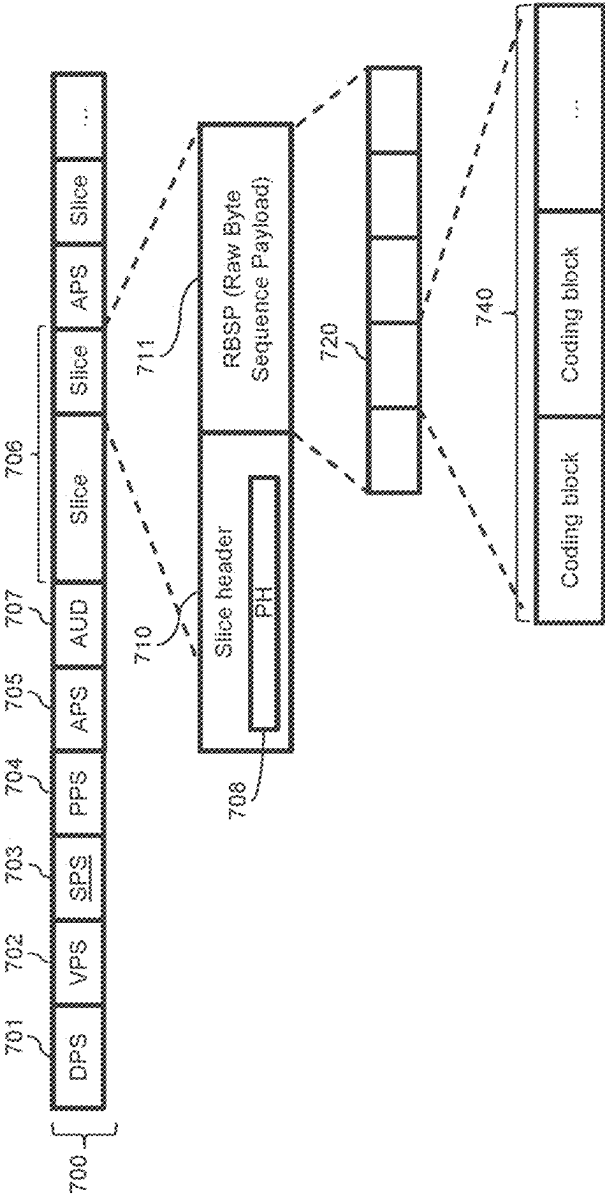


Figure 7

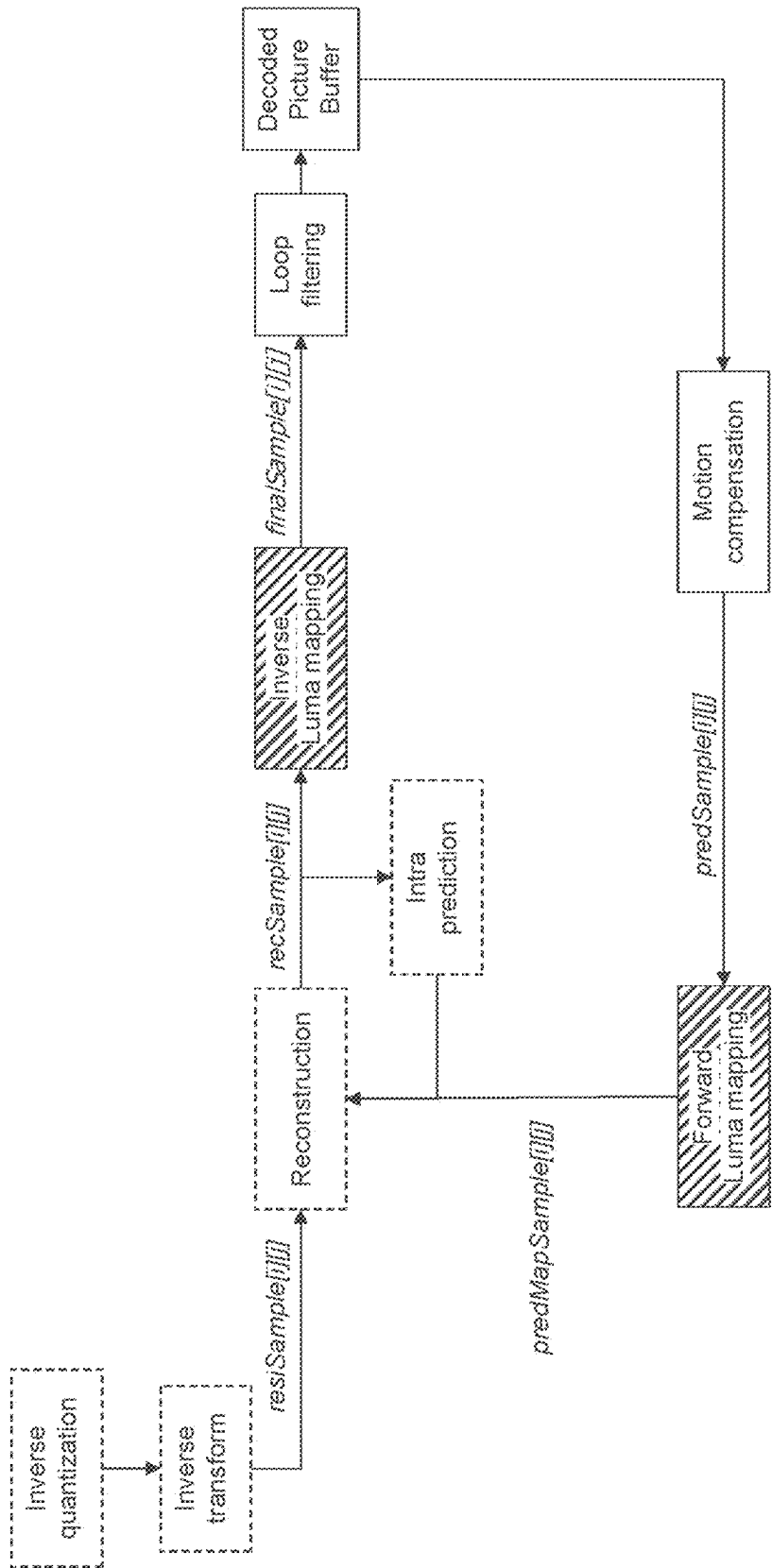


Figure 8

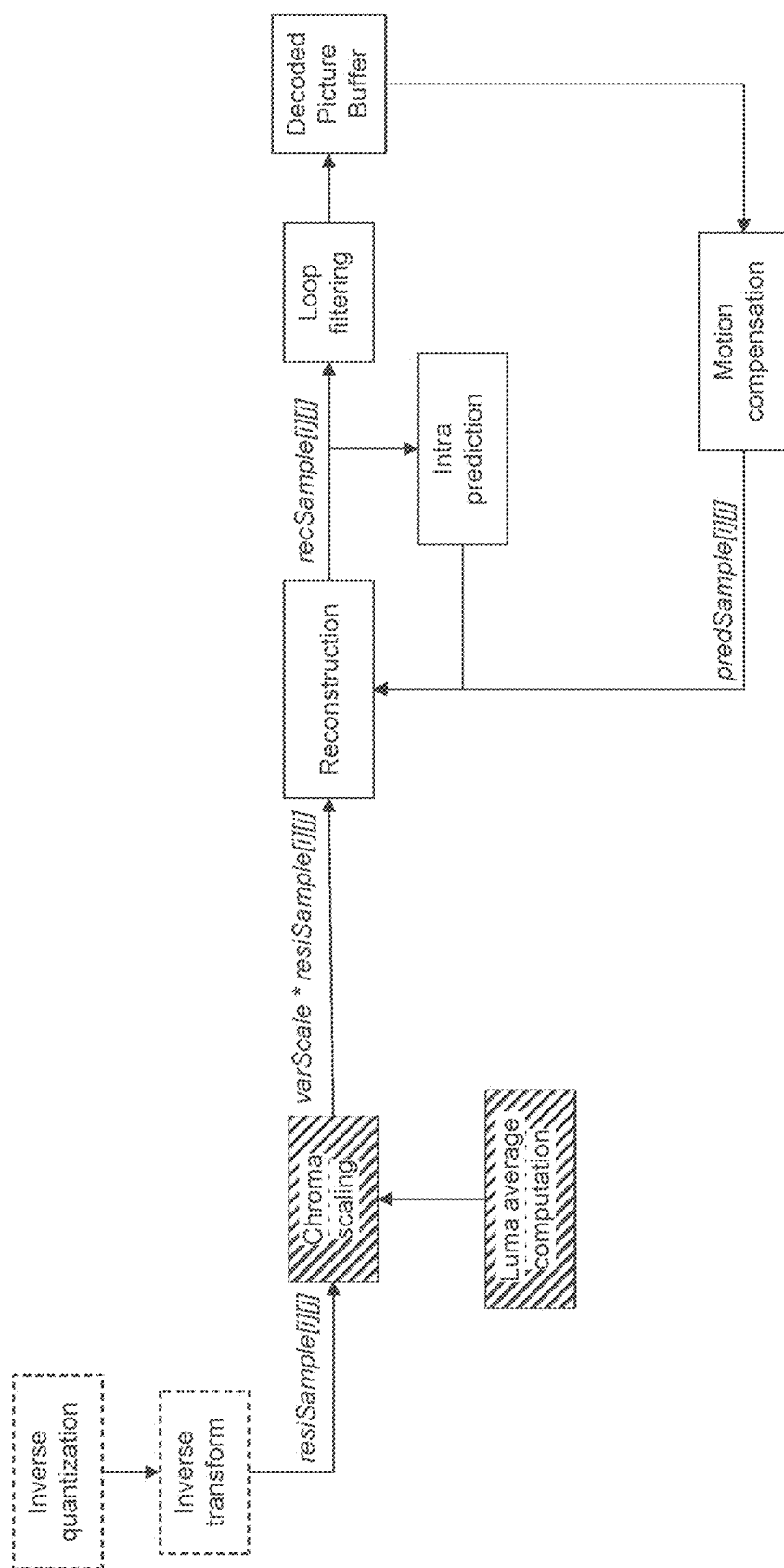


Figure 9

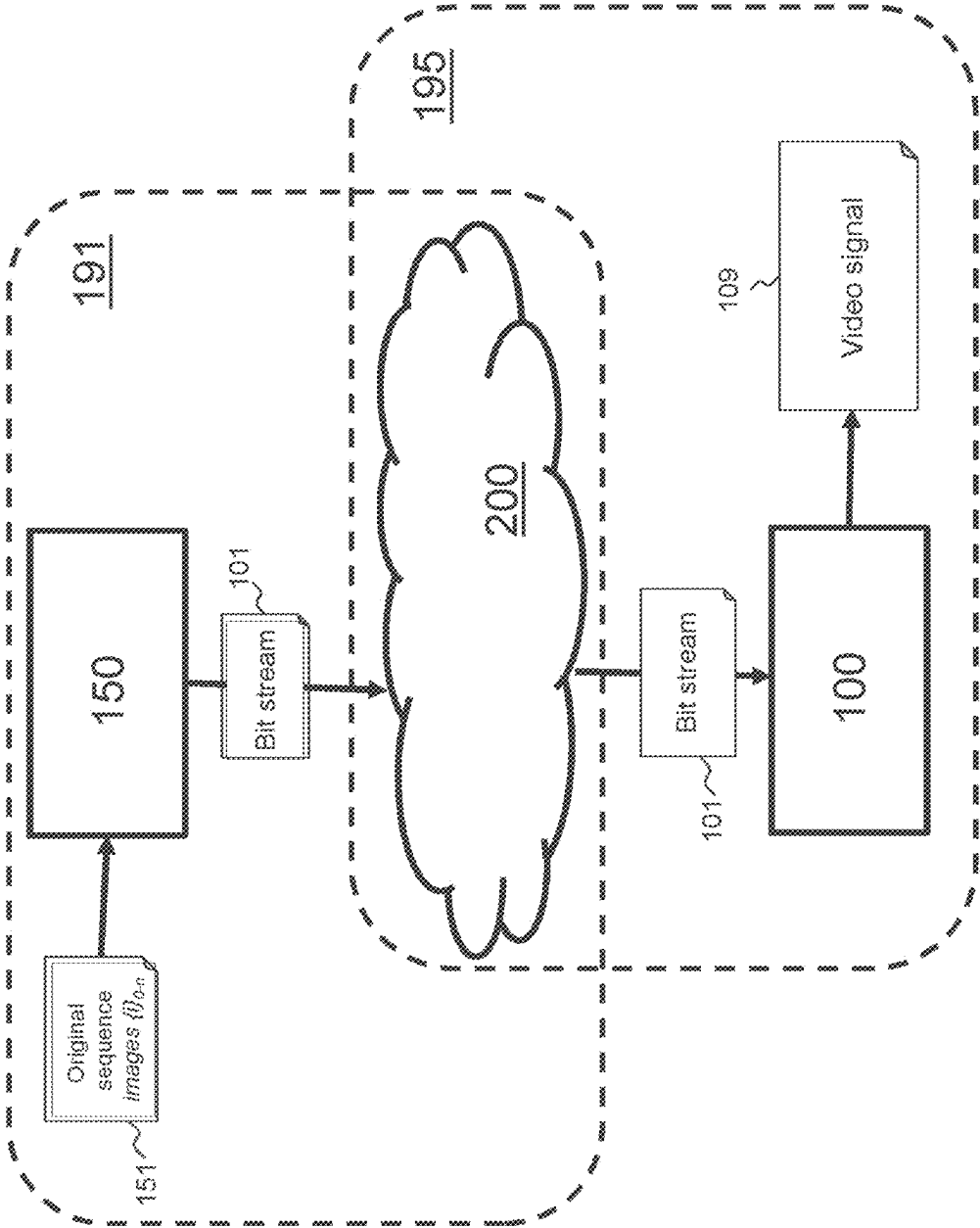


Figure 10

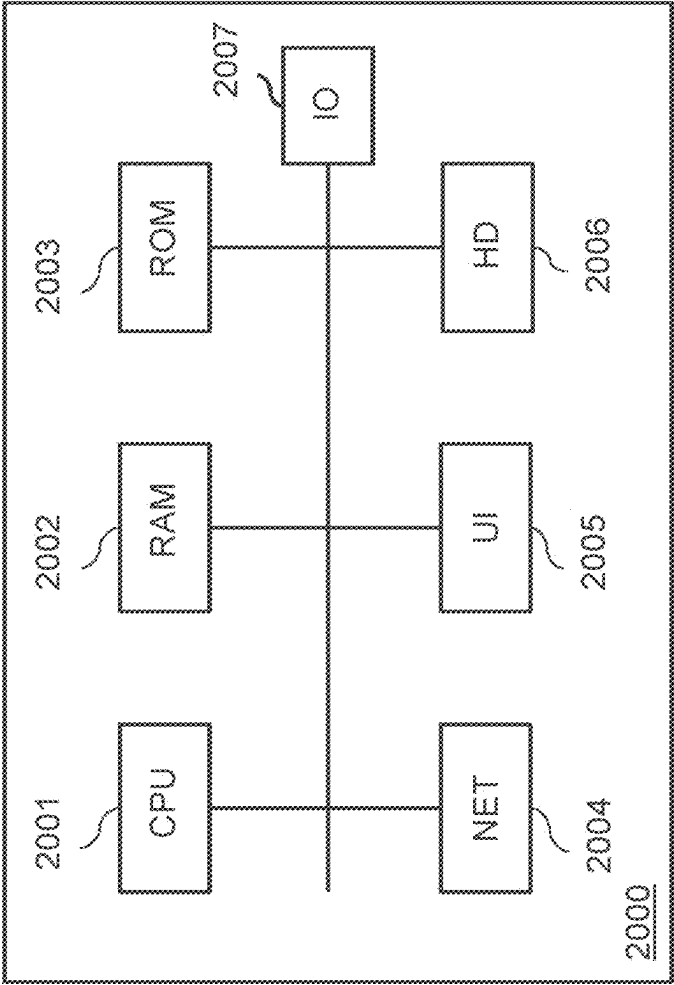


Figure 11

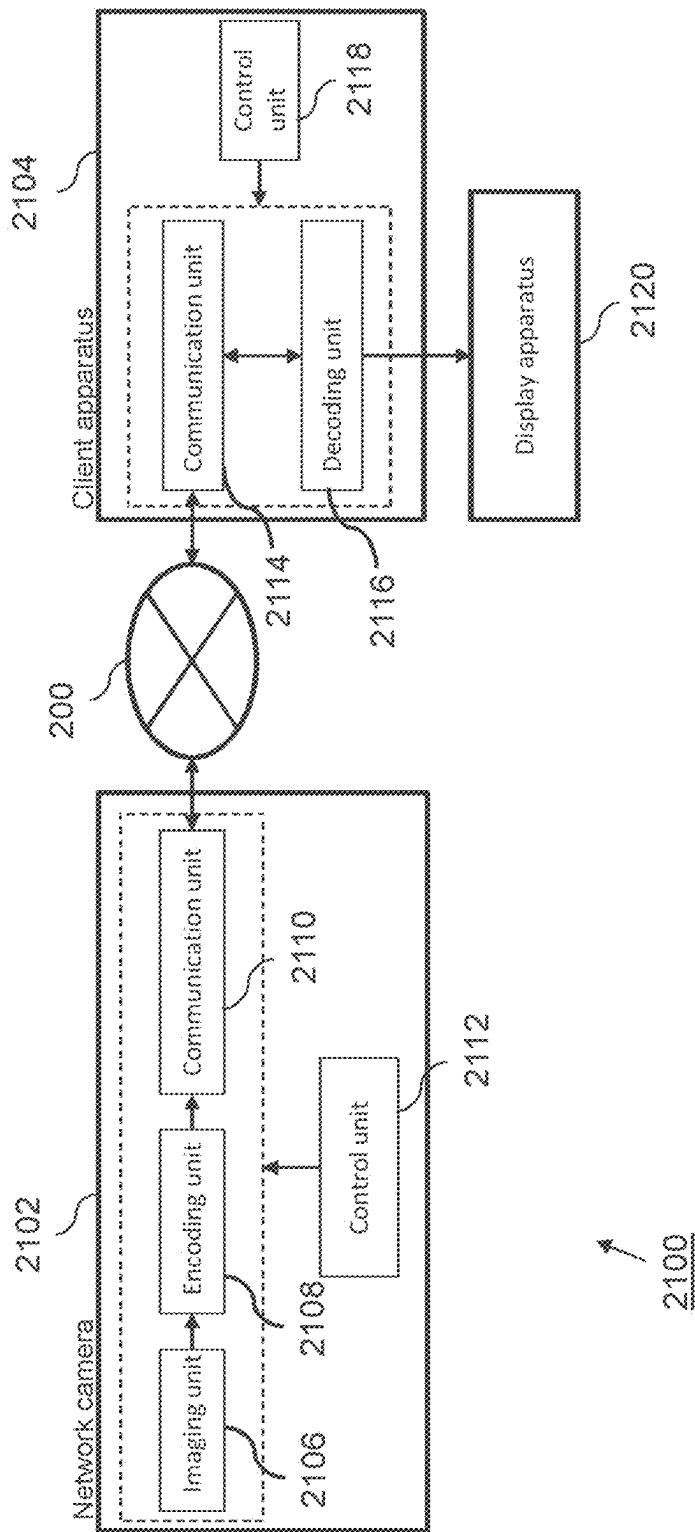


Figure 12

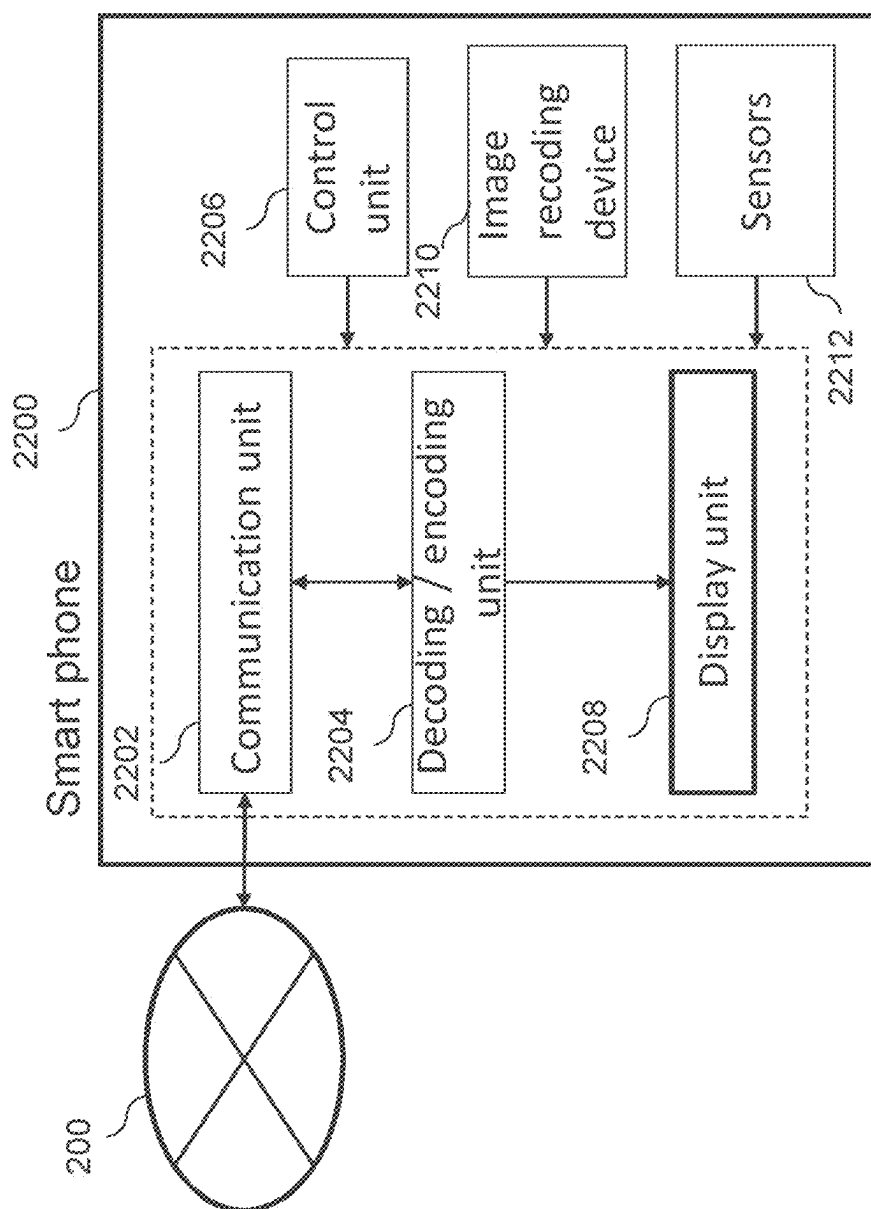


Figure 13

HIGH LEVEL SYNTAX FOR VIDEO CODING AND DECODING

CROSS REFERENCE TO RELATED APPLICATION

[0001] This application is a continuation of U.S. patent application Ser. No. 17/910,745, filed on Sep. 9, 2022, which is the National Phase application of PCT Application No. PCT/EP2021/055661, filed on Mar. 5, 2021, which claims the benefit under 35 U.S.C. § 119(a)-(d) of United Kingdom Patent Application No. 2003562.2, filed on Mar. 11, 2020, and United Kingdom Patent Application No. 2005741.0, filed on Apr. 20, 2020. The above cited patent applications are incorporated herein by reference in their entirety.

FIELD OF INVENTION

[0002] The present invention relates to video coding and decoding, and in particular to the high level syntax used in the bitstream.

BACKGROUND

[0003] Recently, the Joint Video Experts Team (JVET), a collaborative team formed by MPEG and ITU-T Study Group 16's VCEG, commenced work on a new video coding standard referred to as Versatile Video Coding (VVC). The goal of VVC is to provide significant improvements in compression performance over the existing HEVC standard (i.e., typically twice as much as before) and to be completed in 2020. The main target applications and services include—but not limited to—360-degree and high-dynamic-range (HDR) videos. In total, JVET evaluated responses from 32 organizations using formal subjective tests conducted by independent test labs. Some proposals demonstrated compression efficiency gains of typically 40% or more when compared to using HEVC. Particular effectiveness was shown on ultra-high definition (UHD) video test material. Thus, we may expect compression efficiency gains well-beyond the targeted 50% for the final standard.

[0004] The JVET exploration model (JEM) uses all the HEVC tools and has introduced a number of new tools. These changes have necessitated a change to the structure of the bitstream, and in particular to the high-level syntax which can have an impact on the overall bitrate of the bitstream.

SUMMARY

[0005] The present invention relates to an improvement to the high level syntax structure, which leads to a reduction in complexity without any degradation in coding performance.

[0006] According to a first aspect of the invention there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the decoding comprises in a case where information which can be signalled in the picture header or in the slice header is signalled in the picture header, imposing (e.g. applying a constraint) that the picture header is not in the slice header; and decoding said bitstream using said syntax elements.

[0007] Optionally, the decoding further comprises parsing a first syntax element indicating whether said information is

to be signalled in a picture header and permitting the parsing of the information which may be signalled in a slice header and a picture header in only one of the slice header or picture header based on the first syntax element.

[0008] Optionally, the first syntax element is a information in picture parameter set flag or an information in picture header flag.

[0009] Optionally, parsing of the information in the slice header is not permitted in the case that the first syntax element indicates that the information is signalled in the picture header.

[0010] Optionally, the method further comprises parsing a second syntax element indicating whether the picture header is in the slice header or not, wherein it is a requirement of bitstream conformance that the second syntax element indicates that the picture header is not in the slice header in the case that the first syntax element indicates that the information is signalled in the picture header.

[0011] Optionally, the information includes one or more of quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information.

[0012] Optionally, the information includes all information that may be signaled in a picture header and a slice header.

[0013] Optionally, the reference picture list information includes one or more of a slice_collocated_from_l0 flag, a slice_collocated_ref_idx, a ph_collocated_from_l0_flag and a ph_collocated_ref_idx.

[0014] According to a second aspect of the invention there is provided a method of encoding video data into a bitstream, the video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the encoding comprises in a case information which can be signalled in the picture header or in the slice header is signalled in the picture header; signalling that the picture header is not in the slice header, and encoding said video data using said syntax elements.

[0015] Optionally, the encoding further comprises encoding a first syntax element indicating whether said information is to be signalled in a picture header and permitting the encoding of the information which may be signalled in a slice header and a picture header in only one of the slice header or picture header based on the first syntax element.

[0016] Optionally, the first syntax element is an information in picture parameter set flag or an information in picture header flag.

[0017] Optionally, encoding of the information in the slice header is not permitted in the case that the first syntax element indicates that the information is signalled in the picture header.

[0018] Optionally, the method further comprises encoding a second syntax element indicating whether the picture header is in the slice header or not, wherein it is a requirement of bitstream conformance that the second syntax element indicates that the picture header is not in the slice header in the case that the first syntax element indicates that the information is signalled in the picture header.

[0019] Optionally, the information includes one or more of quantization parameter value information, reference picture

list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information.

[0020] Optionally, the information includes all information that may be signaled in a picture header and a slice header.

[0021] Optionally, the reference picture list information includes one or more of a `slice_collocated_from_l0` flag, a `slice_collocated_ref_idx`, a `ph_collocated_from_l0_flag` and a `ph_collocated_ref_idx`.

[0022] In an alternative aspect of the invention there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the decoding comprises in a case where information which can be signalled in the picture header or in the slice header is signalled in the slice header, imposing that the picture header is not in the slice header; and decoding said bitstream using said syntax elements.

[0023] According to another aspect of the present invention, there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the decoding comprises treating as not applicable, in combination, (a) a syntax element indicating that tool information is signalled in a picture header rather than in a slice header and (b) a syntax element indicating that a picture header is signalled in a slice header; and decoding said bitstream using said syntax elements. The decoding does not occur when said syntax elements (a) and (b) treated as not applicable in combination are present.

[0024] According to a related aspect of the present invention, a bitstream comprises: video data corresponding to one or more slices; a picture header comprising syntax elements to be used when decoding one or more slices; and a slice header comprising syntax elements to be used when decoding a slice. The bitstream has a constraint that there must not be present therein, in combination, (a) a syntax element indicating that tool information is signalled in a picture header rather than in a slice header and (b) a syntax element indicating that the (a) picture header is signalled in a slice header. The tool information may be any one of quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information. In a related aspect there is provided a method of decoding said bitstream. In another related aspect there is provided a decoder configured to decode said bitstream. The bitstream may be constrained for conformity with a video coding standard. In an embodiment, said video coding standard is the Versatile Video Coding standard. The constraint may be applied systematically throughout the bitstream. For example, in embodiments, the constraint is applied for any or all of sequences, pictures and slices in a bitstream.

[0025] According to another aspect of the present invention, there is provided a method of decoding video data from a bitstream, the bitstream comprising video data correspond-

ing to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the decoding comprises treating as not applicable, in combination, (a) a syntax element indicating that tool information is signalled in a slice header rather than in a picture header and (b) a syntax element indicating that a picture header is signalled in a slice header; and decoding said bitstream using said syntax elements. The decoding does not occur when said syntax elements (a) and (b) treated as not applicable in combination are present.

[0026] According to another aspect of the present invention, there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, wherein a picture header is to be signalled in a slice header and wherein the parsing of information which may otherwise be signalled in a slice header and a picture header, in only one of the slice header or picture header are constrained; and the decoding comprises in case where a syntax element (`xxx_info_in_ph_flag`), indicates the presence of tool information in the picture header (e.g. when `xxx_info_in_ph_flag=1`), the picture header is not signalled in a slice header (e.g. force `picture_header_in_slice_header_flag` to be 0).

[0027] According to a related aspect of the present invention, a bitstream comprises: video data corresponding to one or more slices; a picture header comprising syntax elements to be used when decoding one or more slices; and a slice header comprising syntax elements to be used when decoding a slice. The bitstream has a constraint that there must not be present therein, in combination, (a) a syntax element indicating that tool information is signalled in a slice header rather than in a picture header and (b) a syntax element indicating that a picture header is signalled in a slice header.

[0028] According to another aspect of the present invention, there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, wherein a picture header is to be signalled in a slice header and wherein the parsing of information which may otherwise be signalled in a slice header and a picture header, in only one of the slice header or picture header are constrained; and the decoding comprises in case where a syntax element (`xxx_info_in_phflag`), indicates the absence of tool information in the picture header (e.g. when `xxx_info_in_ph_flag=0`), the picture header is not signalled in a slice header (e.g. force `picture_header_in_slice_header_flag` to be 0).

[0029] In a first further aspect of the invention, there is provided method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the decoding comprises, in a case where a picture header is to be signalled

in a slice header, permitting parsing of information which may otherwise be signalled in a slice header and a picture header, in only one of the slice header or picture header; and decoding said bitstream using said syntax elements.

[0030] When the picture header is in the slice header this implies only one slice is present for the current picture. Thus, having the information transmitted or transmittable for both slice and picture doesn't increase the flexibility for encoder or decoder as the parameters will be the same. In other words, if the information is in the picture header, corresponding information in the slice header will be redundant. Similarly, if the information is in the slice header, corresponding information in the picture header will be redundant. By only permitting the information to be in the picture header or the slice header, in a case where the picture header is in the slice header, the decoder implementation may be simplified by limiting the redundancies in the signaling. Accordingly, the parsing may be simplified without any loss in coding efficiency.

[0031] The decoding may further comprise parsing a first syntax element indicating whether a picture header is to be signalled in a slice header and permitting the parsing of the information which may be signalled in a slice header and a picture header in only one of the slice header or picture header based on the first syntax element. The first syntax element may be a picture header in slice header flag.

[0032] Optionally, the parsing of the information in the slice header is not permitted in the case that the first syntax element indicates that the picture header is signalled in the slice header. A second syntax element may be parsed indicating whether the information is in a picture header or not, wherein it is a requirement of bitstream conformance that the second syntax element indicates that the information is signalled in the picture header in the case that the first syntax element indicates that the picture header is signalled in the slice header.

[0033] Alternatively, the signalling of the information in the picture header is not permitted in the case that the first syntax element indicates that the picture header is signalled in the slice header. The method may further comprise parsing a second syntax element indicating whether the information is in a picture header or not, wherein it is a requirement of bitstream conformance that the second syntax element indicates that the information is in the picture header in the case that the first syntax element indicates that the picture header is signalled in the slice header. The second syntax element may be an information in picture parameter set flag, wherein the information is in the picture header when the flag is set and the information is in the slice header or not present when not set.

[0034] According to embodiments, the information may include one or more of quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information. For example, all of the quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information.

[0035] Optionally, the information includes all information that may be signaled in a picture header and a slice header.

[0036] The reference picture list information may include one or more of the following syntax elements slice_collocated_from_10 flag, a slice_collocated_ref_idx, a ph_collocated_from_10_flag and a ph_collocated_ref_idx.

[0037] A number of weights for weighted prediction that may be parsed may be restricted in the case where a picture header is to be signalled in a slice header.

[0038] According to a second further aspect of the invention, there is provided a method of encoding video data into a bitstream, the video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the encoding comprises in a case where a picture header is to be signalled in a slice header, permitting encoding of information which may otherwise be signalled in a slice header and a picture header, in only one of the slice header or picture header, and encoding said video data using said syntax elements.

[0039] When the picture header is in the slice header this implies only one slice is present for the current picture. Thus, having the information transmitted or transmittable for both slice and picture doesn't increase the flexibility for encoder or decoder as the parameters will be the same. In other words, if the information is in the picture header, corresponding information in the slice header will be redundant. Similarly, if the information is in the slice header, corresponding information in the picture header will be redundant. By only permitting the information to be in the picture header or the slice header, in a case where the picture header is in the slice header, the decoder implementation may be simplified by limiting the redundancies in the signaling. Accordingly, the encoding may be simplified without any loss in coding efficiency and the signalling cost reduced (because the relevant information is only included once in the bitstream).

[0040] The encoding may further comprise encoding a first syntax element indicating whether a picture header is to be signalled in a slice header and permitting the encoding of the information which may be signalled in a slice header and a picture header in only one of the slice header or picture header based on the first syntax element.

[0041] The first syntax element may be a picture header in slice header flag.

[0042] Encoding the information in the slice header may not be permitted in the case that the first syntax element indicates that the picture header is signalled in the slice header.

[0043] A second syntax element may be encoded indicating whether the information is in a picture header or not, wherein it is a requirement of bitstream conformance that the second syntax element indicates that the information is in the picture header in the case that the first syntax element indicates that the picture header is signalled in the slice header.

[0044] Alternatively, signalling of the information in the picture header is not permitted in the case that the first syntax element indicates that the picture header is signalled in the slice header.

[0045] A second syntax element may be encoded indicating whether the information is in a picture header or not, wherein it is a requirement of bitstream conformance that the second syntax element indicates that the information is

in the picture header in the case that the first syntax element indicates that the picture header is signalled in the slice header.

[0046] The second syntax element may be an information in picture parameter set flag or an information in picture header flag, wherein the information is signalled in the picture header when the flag is set and the information is signalled in the slice header or not present when not set.

[0047] The information may include one or more of quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information. Optionally, the information includes all information that may be signalled in a picture header and a slice header. For example, all of the quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop filtering (ALF) information.

[0048] The reference picture list information includes one or more of a slice_collocated_from_l0 flag, a slice_collocated_ref_idx, a ph_collocated_from_l0_flag and a ph_collocated_ref_idx.

[0049] Optionally, a number of weights for weighted prediction may be restricted in the case where a picture header is to be signalled in a slice header.

[0050] In a third further aspect according to the invention, there is provided a method of decoding a bitstream containing video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and said method comprising parsing, in the slice header a syntax element indicating whether a picture header is signalled in a slice header, wherein ALF APS ID related syntax element is parsed prior to the syntax element indicating whether a picture header is signalled in a slice header. The ALF APS ID related information may be parsed near or at the beginning of the slice header.

[0051] In a fourth further aspect according to the invention there is provided a method of encoding a video data comprising one or more slices into a bitstream, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and said method comprises parsing, in the slice header a syntax element indicating whether a picture header is signalled in a slice header; wherein ALF APS ID related syntax element is encoded prior to the syntax element indicating whether a picture header is signalled in a slice header. The ALF APS ID related information may be encoded near or at the beginning of the slice header.

[0052] In a fifth further aspect of the invention, there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the decoding comprises, in a case where a picture header is to be signalled in a slice header, restricting a number of weights signalled for a weighted prediction mode; and decoding said bitstream using said syntax elements. In a case where a picture header

is to be signalled in a slice header, parsing of information which may otherwise be signalled in a slice header and a picture header may be permitted in only one of the slice header or picture header.

[0053] In a sixth further aspect according to the invention, there is provided a method of encoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the encoding comprises, in a case where a picture header is to be signalled in a slice header, restricting a number of weights encoded for a weighted prediction mode; and encoding said bitstream using said syntax elements. In a case where a picture header is to be signalled in a slice header, encoding of information which may otherwise be signalled in a slice header and a picture header, may be permitted in only one of the slice header or picture header.

[0054] According to a seventh further aspect of the invention, there is provided a decoder for decoding video data from a bitstream, the decoder being configured to perform the method of any of the first, third or fifth further aspects.

[0055] According to an eighth further aspect of the invention there is provided an encoder for encoding video data into a bitstream, the encoder being configured to perform the method of any of second, fourth or sixth further aspects.

[0056] According to a ninth further aspect of the invention, there is provided a computer program which upon execution causes the method of any of the first to sixth further aspects to be performed. The program may be provided on its own or may be carried on, by or in a carrier medium. The carrier medium may be non-transitory, for example a storage medium, in particular a computer-readable storage medium. The carrier medium may also be transitory, for example a signal or other transmission medium. The signal may be transmitted via any suitable network, including the Internet. Further features of the invention are characterised by the independent and dependent claims.

[0057] According to a first yet further aspect there is provided a method of decoding video data from a bitstream, the bitstream comprising video data corresponding to one or more slices, wherein the bitstream comprises a picture header comprising syntax elements to be used when decoding one or more slices, and a slice header comprising syntax elements to be used when decoding a slice, and the bitstream is constrained so that in a case where the bitstream includes a first syntax element having a value indicating that information which can be signalled in the picture header or in the slice header is signalled in the picture header, the bitstream also includes a second syntax element having a value indicating that the picture header is not in the slice header, the method comprising: decoding said bitstream using said syntax elements. The bitstream may be constrained for conformity with a video coding standard. In an embodiment, said video coding standard is the Versatile Video Coding standard. The second syntax element may be a picture header in slice header syntax element. The first syntax element may be a flag indicating the signalling in the picture header of one or more of quantization parameter value information, reference picture list information, deblocking filter information, sample adaptive offset (SAO) information, weighted prediction information and adaptive loop

filtering (ALF) information. The bitstream constraint may be applied systematically. For example, in embodiments, the constraint is applied for any or all of sequences, pictures and slices in a bitstream.

[0058] According to a second yet further aspect, there is provided a method for encoding or decoding video data into or from a bitstream, said method comprising applying a constraint relating to whether a picture header is permitted in a slice header based on information which can be signalled in the picture header or in the slice header is signalled in the picture header. According to a third yet further aspect, there is provided a device configured to perform the method of the second yet further aspect. According to a fourth yet further aspect, there is provided a computer program comprising instructions which upon execution cause the method of the second yet further aspect to be performed.

[0059] Any feature in one aspect of the invention may be applied to other aspects of the invention, in any appropriate combination. In particular, method aspects may be applied to apparatus aspects, and vice versa.

[0060] Furthermore, features implemented in hardware may be implemented in software, and vice versa. Any reference to software and hardware features herein should be construed accordingly. Any apparatus feature as described herein may also be provided as a method feature, and vice versa. As used herein, means plus function features may be expressed alternatively in terms of their corresponding structure, such as a suitably programmed processor and associated memory.

[0061] It should also be appreciated that particular combinations of the various features described and defined in any aspects of the invention can be implemented and/or supplied and/or used independently.

BRIEF DESCRIPTION OF THE DRAWINGS

[0062] Reference will now be made, by way of example, to the accompanying drawings, in which:

[0063] FIG. 1 is a diagram for use in explaining a coding structure used in HEVC and VVC;

[0064] FIG. 2 is a block diagram schematically illustrating a data communication system in which one or more embodiments of the invention may be implemented;

[0065] FIG. 3 is a block diagram illustrating components of a processing device in which one or more embodiments of the invention may be implemented;

[0066] FIG. 4 is a flow chart illustrating steps of an encoding method according to embodiments of the invention;

[0067] FIG. 5 is a flow chart illustrating steps of a decoding method according to embodiments of the invention;

[0068] FIG. 6 illustrates the structure of the bitstream in the exemplary coding system VVC

[0069] FIG. 7 illustrates another structure of the bitstream in the exemplary coding system VVC

[0070] FIG. 8 illustrates Luma Modelling Chroma Scaling (LMCS);

[0071] FIG. 9 shows a sub tool of LMCS;

[0072] FIG. 10 is a diagram showing a system comprising an encoder or a decoder and a communication network according to embodiments of the present invention;

[0073] FIG. 11 is a schematic block diagram of a computing device for implementation of one or more embodiments of the invention;

[0074] FIG. 12 is a diagram illustrating a network camera system; and

[0075] FIG. 13 is a diagram illustrating a smart phone.

DETAILED DESCRIPTION

[0076] FIG. 1 relates to a coding structure used in the High Efficiency Video Coding (HEVC) video standard. A video sequence **1** is made up of a succession of digital images **i**. Each such digital image is represented by one or more matrices. The matrix coefficients represent pixels.

[0077] An image **2** of the sequence may be divided into slices **3**. A slice may in some instances constitute an entire image. These slices are divided into non-overlapping Coding Tree Units (CTUs). A Coding Tree Unit (CTU) is the basic processing unit of the High Efficiency Video Coding (HEVC) video standard and conceptually corresponds in structure to macroblock units that were used in several previous video standards. A CTU is also sometimes referred to as a Largest Coding Unit (LCU). A CTU has luma and chroma component parts, each of which component parts is called a Coding Tree Block (CTB). These different color components are not shown in FIG. 1.

[0078] A CTU is generally of size 64 pixels×64 pixels. Each CTU may in turn be iteratively divided into smaller variable-size Coding Units (CUs) **5** using a quadtree decomposition.

[0079] Coding units are the elementary coding elements and are constituted by two kinds of sub-unit called a Prediction Unit (PU) and a Transform Unit (TU). The maximum size of a PU or TU is equal to the CU size. A Prediction Unit corresponds to the partition of the CU for prediction of pixels values. Various different partitions of a CU into PUs are possible as shown by **606** including a partition into 4 square PUs and two different partitions into 2 rectangular PUs. A Transform Unit is an elementary unit that is subjected to spatial transformation using DCT. A CU can be partitioned into TUs based on a quadtree representation **607**.

[0080] Each slice is embedded in one Network Abstraction Layer (NAL) unit. In addition, the coding parameters of the video sequence are stored in dedicated NAL units called parameter sets. In HEVC and H.264/AVC two kinds of parameter sets NAL units are employed: first, a Sequence Parameter Set (SPS) NAL unit that gathers all parameters that are unchanged during the whole video sequence. Typically, it handles the coding profile, the size of the video frames and other parameters. Secondly, a Picture Parameter Set (PPS) NAL unit includes parameters that may change from one image (or frame) to another of a sequence. HEVC also includes a Video Parameter Set (VPS) NAL unit which contains parameters describing the overall structure of the bitstream. The VPS is a new type of parameter set defined in HEVC, and applies to all of the layers of a bitstream. A layer may contain multiple temporal sub-layers, and all version 1 bitstreams are restricted to a single layer. HEVC has certain layered extensions for scalability and multiview and these will enable multiple layers, with a backwards compatible version 1 base layer.

[0081] FIG. 2 illustrates a data communication system in which one or more embodiments of the invention may be implemented. The data communication system comprises a transmission device, in this case a server **201**, which is operable to transmit data packets of a data stream to a receiving device, in this case a client terminal **202**, via a data

communication network **200**. The data communication network **200** may be a Wide Area Network (WAN) or a Local Area Network (LAN). Such a network may be for example a wireless network (Wifi/802.11a or b or g), an Ethernet network, an Internet network or a mixed network composed of several different networks. In a particular embodiment of the invention the data communication system may be a digital television broadcast system in which the server **201** sends the same data content to multiple clients.

[0082] The data stream **204** provided by the server **201** may be composed of multimedia data representing video and audio data. Audio and video data streams may, in some embodiments of the invention, be captured by the server **201** using a microphone and a camera respectively. In some embodiments data streams may be stored on the server **201** or received by the server **201** from another data provider, or generated at the server **201**. The server **201** is provided with an encoder for encoding video and audio streams in particular to provide a compressed bitstream for transmission that is a more compact representation of the data presented as input to the encoder.

[0083] In order to obtain a better ratio of the quality of transmitted data to quantity of transmitted data, the compression of the video data may be for example in accordance with the HEVC format or H.264/AVC format.

[0084] The client **202** receives the transmitted bitstream and decodes the reconstructed bitstream to reproduce video images on a display device and the audio data by a loud speaker.

[0085] Although a streaming scenario is considered in the example of FIG. 2, it will be appreciated that in some embodiments of the invention the data communication between an encoder and a decoder may be performed using for example a media storage device such as an optical disc.

[0086] In one or more embodiments of the invention a video image is transmitted with data representative of compensation offsets for application to reconstructed pixels of the image to provide filtered pixels in a final image.

[0087] FIG. 3 schematically illustrates a processing device **300** configured to implement at least one embodiment of the present invention. The processing device **300** may be a device such as a micro-computer, a workstation or a light portable device. The device **300** comprises a communication bus **313** connected to:

[0088] a central processing unit **311**, such as a micro-processor, denoted CPU;

[0089] a read only memory **306**, denoted ROM, for storing computer programs for implementing the invention;

[0090] a random access memory **312**, denoted RAM, for storing the executable code of the method of embodiments of the invention as well as the registers adapted to record variables and parameters necessary for implementing the method of encoding a sequence of digital images and/or the method of decoding a bitstream according to embodiments of the invention; and

[0091] a communication interface **302** connected to a communication network **303** over which digital data to be processed are transmitted or received

[0092] Optionally, the apparatus **300** may also include the following components:

[0093] a data storage means **304** such as a hard disk, for storing computer programs for implementing methods of one or more embodiments of the invention and data

used or produced during the implementation of one or more embodiments of the invention;

[0094] a disk drive **305** for a disk **306**, the disk drive being adapted to read data from the disk **306** or to write data onto said disk;

[0095] a screen **309** for displaying data and/or serving as a graphical interface with the user, by means of a keyboard **310** or any other pointing means.

[0096] The apparatus **300** can be connected to various peripherals, such as for example a digital camera **320** or a microphone **308**, each being connected to an input/output card (not shown) so as to supply multimedia data to the apparatus **300**.

[0097] The communication bus provides communication and interoperability between the various elements included in the apparatus **300** or connected to it. The representation of the bus is not limiting and in particular the central processing unit is operable to communicate instructions to any element of the apparatus **300** directly or by means of another element of the apparatus **300**.

[0098] The disk **306** can be replaced by any information medium such as for example a compact disk (CD-ROM), rewritable or not, a ZIP disk or a memory card and, in general terms, by an information storage means that can be read by a microcomputer or by a microprocessor, integrated or not into the apparatus, possibly removable and adapted to store one or more programs whose execution enables the method of encoding a sequence of digital images and/or the method of decoding a bitstream according to the invention to be implemented.

[0099] The executable code may be stored either in read only memory **306**, on the hard disk **304** or on a removable digital medium such as for example a disk **306** as described previously. According to a variant, the executable code of the programs can be received by means of the communication network **303**, via the interface **302**, in order to be stored in one of the storage means of the apparatus **300** before being executed, such as the hard disk **304**.

[0100] The central processing unit **311** is adapted to control and direct the execution of the instructions or portions of software code of the program or programs according to the invention, instructions that are stored in one of the aforementioned storage means. On powering up, the program or programs that are stored in a non-volatile memory, for example on the hard disk **304** or in the read only memory **306**, are transferred into the random access memory **312**, which then contains the executable code of the program or programs, as well as registers for storing the variables and parameters necessary for implementing the invention.

[0101] In this embodiment, the apparatus is a programmable apparatus which uses software to implement the invention. However, alternatively, the present invention may be implemented in hardware (for example, in the form of an Application Specific Integrated Circuit or ASIC).

[0102] FIG. 4 illustrates a block diagram of an encoder according to at least one embodiment of the invention. The encoder is represented by connected modules, each module being adapted to implement, for example in the form of programming instructions to be executed by the CPU **311** of device **300**, at least one corresponding step of a method implementing at least one embodiment of encoding an image of a sequence of images according to one or more embodiments of the invention.

[0103] An original sequence of digital images $i0$ to in **401** is received as an input by the encoder **400**. Each digital image is represented by a set of samples, known as pixels.

[0104] A bitstream **410** is output by the encoder **400** after implementation of the encoding process. The bitstream **410** comprises a plurality of encoding units or slices, each slice comprising a slice header for transmitting encoding values of encoding parameters used to encode the slice and a slice body, comprising encoded video data.

[0105] The input digital images $i0$ to in **401** are divided into blocks of pixels by module **402**. The blocks correspond to image portions and may be of variable sizes (e.g. 4×4 , 8×8 , 16×16 , 32×32 , 64×64 , 128×128 pixels and several rectangular block sizes can be also considered). A coding mode is selected for each input block. Two families of coding modes are provided: coding modes based on spatial prediction coding (Intra prediction), and coding modes based on temporal prediction (Inter coding, Merge, SKIP). The possible coding modes are tested.

[0106] Module **403** implements an Intra prediction process, in which the given block to be encoded is predicted by a predictor computed from pixels of the neighbourhood of said block to be encoded. An indication of the selected Intra predictor and the difference between the given block and its predictor is encoded to provide a residual if the Intra coding is selected.

[0107] Temporal prediction is implemented by motion estimation module **404** and motion compensation module **405**. Firstly, a reference image from among a set of reference images **416** is selected, and a portion of the reference image, also called reference area or image portion, which is the closest area to the given block to be encoded, is selected by the motion estimation module **404**. Motion compensation module **405** then predicts the block to be encoded using the selected area. The difference between the selected reference area and the given block, also called a residual block, is computed by the motion compensation module **405**. The selected reference area is indicated by a motion vector.

[0108] Thus, in both cases (spatial and temporal prediction), a residual is computed by subtracting the prediction from the original block.

[0109] In the INTRA prediction implemented by module **403**, a prediction direction is encoded. In the temporal prediction, at least one motion vector is encoded. In the Inter prediction implemented by modules **404**, **405**, **416**, **418**, **417**, at least one motion vector or data for identifying such motion vector is encoded for the temporal prediction.

[0110] Information relative to the motion vector and the residual block is encoded if the Inter prediction is selected. To further reduce the bitrate, assuming that motion is homogeneous, the motion vector is encoded by difference with respect to a motion vector predictor. Motion vector predictors of a set of motion information predictors is obtained from the motion vectors field **418** by a motion vector prediction and coding module **417**.

[0111] The encoder **400** further comprises a selection module **406** for selection of the coding mode by applying an encoding cost criterion, such as a rate-distortion criterion. In order to further reduce redundancies a transform (such as DCT) is applied by transform module **407** to the residual block, the transformed data obtained is then quantized by quantization module **408** and entropy encoded by entropy

encoding module **409**. Finally, the encoded residual block of the current block being encoded is inserted into the bitstream **410**.

[0112] The encoder **400** also performs decoding of the encoded image in order to produce a reference image for the motion estimation of the subsequent images. This enables the encoder and the decoder receiving the bitstream to have the same reference frames. The inverse quantization module **411** performs inverse quantization of the quantized data, followed by an inverse transform by reverse transform module **412**. The reverse intra prediction module **413** uses the prediction information to determine which predictor to use for a given block and the reverse motion compensation module **414** actually adds the residual obtained by module **412** to the reference area obtained from the set of reference images **416**.

[0113] Post filtering is then applied by module **415** to filter the reconstructed frame of pixels. In the embodiments of the invention an SAO loop filter is used in which compensation offsets are added to the pixel values of the reconstructed pixels of the reconstructed image

[0114] FIG. 5 illustrates a block diagram of a decoder **60** which may be used to receive data from an encoder according an embodiment of the invention. The decoder is represented by connected modules, each module being adapted to implement, for example in the form of programming instructions to be executed by the CPU **311** of device **300**, a corresponding step of a method implemented by the decoder **60**.

[0115] The decoder **60** receives a bitstream **61** comprising encoding units, each one being composed of a header containing information on encoding parameters and a body containing the encoded video data. The structure of the bitstream in VVC is described in more detail below with reference to FIG. 6. As explained with respect to FIG. 4, the encoded video data is entropy encoded, and the motion vector predictors' indexes are encoded, for a given block, on a predetermined number of bits. The received encoded video data is entropy decoded by module **62**. The residual data are then dequantized by module **63** and then a reverse transform is applied by module **64** to obtain pixel values.

[0116] The mode data indicating the coding mode are also entropy decoded and based on the mode, an INTRA type decoding or an INTER type decoding is performed on the encoded blocks of image data.

[0117] In the case of INTRA mode, an INTRA predictor is determined by intra reverse prediction module **65** based on the intra prediction mode specified in the bitstream.

[0118] If the mode is INTER, the motion prediction information is extracted from the bitstream so as to find the reference area used by the encoder. The motion prediction information is composed of the reference frame index and the motion vector residual. The motion vector predictor is added to the motion vector residual in order to obtain the motion vector by motion vector decoding module **70**.

[0119] Motion vector decoding module **70** applies motion vector decoding for each current block encoded by motion prediction. Once an index of the motion vector predictor, for the current block has been obtained the actual value of the motion vector associated with the current block can be decoded and used to apply reverse motion compensation by module **66**.

[0120] The reference image portion indicated by the decoded motion vector is extracted from a reference image

68 to apply the reverse motion compensation 66. The motion vector field data 71 is updated with the decoded motion vector in order to be used for the inverse prediction of subsequent decoded motion vectors.

[0121] Finally, a decoded block is obtained. Post filtering is applied by post filtering module 67. A decoded video signal 69 is finally provided by the decoder 60.

[0122] FIG. 6 illustrates the organisation of the bitstream in the exemplary coding system VVC as describe in JVET-Q2001-vD.

[0123] A bitstream 61 according to the VVC coding system is composed of an ordered sequence of syntax elements and coded data. The syntax elements and coded data are placed into Network Abstraction Layer (NAL) units 601-608. There are different NAL unit types. The network abstraction layer provides the ability to encapsulate the bitstream into different protocols, like RTP/IP, standing for Real Time Protocol/Internet Protocol, ISO Base Media File Format, etc. The network abstraction layer also provides a framework for packet loss resilience.

[0124] NAL units are divided into Video Coding Layer (VCL) NAL units and non-VCL NAL units. The VCL NAL units contain the actual encoded video data. The non-VCL NAL units contain additional information. This additional information may be parameters needed for the decoding of the encoded video data or supplemental data that may enhance usability of the decoded video data. NAL units 606 correspond to slices and constitute the VCL NAL units of the bitstream.

[0125] Different NAL units 601-605 correspond to different parameter sets, these NAL units are non-VCL NAL units. The Decoder Parameter Set (DPS) NAL unit 301 contains parameters that are constant for a given decoding process. The Video Parameter Set (VPS) NAL unit 602 contains parameters defined for the whole video, and thus the whole bitstream. The DPS NAL unit may define parameters more static than the parameters in the VPS. In other words, the parameters of DPS change less frequently than the parameter of the VPS.

[0126] The Sequence Parameter Set (SPS) NAL unit 603 contains parameters defined for a video sequence. In particular, the SPS NAL unit may define the sub pictures layout and associated parameters of the video sequences. The parameters associated to each subpicture specifies the coding constraints applied to the subpicture. In particular, it comprises a flag indicating that the temporal prediction between subpictures is restricted to the data coming from the same subpicture. Another flag may enable or disable the loop filters across the subpicture boundaries.

[0127] The Picture Parameter Set (PPS) NAL unit 604, PPS contains parameters defined for a picture or a group of pictures. The Adaptation Parameter Set (APS) NAL unit 605, contains parameters for loop filters typically the Adaptive Loop Filter (ALF) or the reshapar model (or luma mapping with chroma scaling (LMCS) model) or the scaling matrices that are used at the slice level.

[0128] The syntax of the PPS as proposed in the current version of VVC comprises syntax elements that specifies the size of the picture in luma samples and also the partitioning of each picture in tiles and slices.

[0129] The PPS contains syntax elements that make it possible to determine the slices location in a frame. Since a subpicture forms a rectangular region in the frame, it is possible to determine the set of slices, the parts of tiles or the

tiles that belong to a subpicture from the Parameter Sets NAL units. The PPS as with the APS have an ID mechanism to limit the amount of same PPS's transmitted.

[0130] The main difference between the PPS and Picture Header is it transmission, the PPS is generally transmitted for a group of pictures compared to the PH which is systematically transmitted for each Picture. Accordingly, the PPS compared to the PH contains parameters which can be constant for several picture.

[0131] The bitstream may also contain Supplemental Enhancement Information (SEI) NAL units (not represented in FIG. 6). The periodicity of occurrence of these parameter sets in the bitstream is variable. A VPS that is defined for the whole bitstream may occur only once in the bitstream. To the contrary, an APS that is defined for a slice may occur once for each slice in each picture. Actually, different slices may rely on the same APS and thus there are generally fewer APS than slices in each picture. In particular, the APS is defined in the picture header. Yet, the ALF APS can be refined in the slice header.

[0132] The Access Unit Delimiter (AUD) NAL unit 607 separates two access units. An access unit is a set of NAL units which may comprise one or more coded pictures with the same decoding timestamp. This optional NAL unit contains only one syntax element in current VVC specification: `pic_type`, this syntax element, indicates that the `slice_type` values for all slices of the coded pictures in the AU. If `pic_type` is set equal to 0, the AU contain only Intra slice. If equal to 1, it contains P and I slices. If equal to 2 it contains B, P or Intra slice

[0133] This NAL unit contains only one syntax element the `pic_type`.

TABLE 1

Syntax AUD	
Descriptor	
<pre> access_unit_delimiter_rbsp() { pic_type u(3) rbsp_trailing_bits() } </pre>	

[0134] In JVET-Q2001-vD the `pic_type` is defined as follow:

[0135] “`pic_type` indicates that the `slice_type` values for all slices of the coded pictures in the AU containing the AU delimiter NAL unit are members of the set listed in Table 2 for the given value of `pic_type`. The value of `pic_type` shall be equal to 0, 1 or 2 in bitstreams conforming to this version of this Specification. Other values of `pic_type` are reserved for future use by ITU-T ISO/IEC. Decoders conforming to this version of this Specification shall ignore reserved values of `pic_type`.”

[0136] The `rbsp_trailing_bits( )` is a function which adds bits in order to be aligned to the end of a byte.

[0137] So after, this function, the amount of bitstream parsed is an integer number of bytes.

TABLE 2

Interpretation of pic_type	
pic_type	slice_type values that may be present in the AU
0	I
1	P, I
2	B, P, I

[0138] The PH NAL unit **608** is the Picture Header NAL unit which groups parameters common to a set of slices of one coded picture. The picture may refer to one or more APS to indicate the AFL parameters, reshaper model and the scaling matrices used by the slices of the Picture.

[0139] Each of the VCL NAL units **606** contains a slice. A slice may correspond to the whole picture or sub picture, a single tile or a plurality of tiles or a fraction of a tile. For example the slice of the FIG. 3 contains several tiles **620**. A slice is composed of a slice header **610** and a raw byte sequence payload, RBSP **611** that contains the coded pixels data encoded as coded blocks **640**.

[0140] The syntax of the PPS as proposed in the current version of VVC comprises syntax elements that specifies the size of the picture in luma samples and also the partitioning of each picture in tiles and slices.

[0141] The PPS contains syntax elements that make it possible to determine the slices location in a frame. Since a subpicture forms a rectangular region in the frame, it is possible to determine the set of slices, the parts of tiles or the tiles that belong to a subpicture from the Parameter Sets NAL units.

NAL Unit Slice

[0142] The NAL unit slice layer contains the slice header and the slice data as illustrated in Table 3.

TABLE 3

Slice layer syntax	
	Descriptor
<pre> slice_layer_rbsp() { slice_header() slice_data() rbsp_slice_trailing_bits() }</pre>	

APS

[0143] The Adaptation Parameter Set (APS) NAL unit **605**, is defined in Table 4 showing the syntax elements.

[0144] As depicted in table Table 4, there are 3 possible types of APS given by the aps_params_type syntax element:

[0145] ALF_AP: for the ALF parameters

[0146] LMCS_APS for the LMCS parameters

[0147] SCALING_APS for Scaling list relative parameters

TABLE 4

Adaptation parameter set syntax	
	Descriptor
adaptation_parameter_set_rbsp() {	
adaptation_parameter_set_id	u(5)
aps_params_type	u(3)
if(aps_params_type == ALF_APS)	
alf_data()	
else if(aps_params_type == LMCS_APS)	
lmcs_data()	
else if(aps_params_type == SCALING_APS)	
scaling_list_data()	
aps_extension_flag	u(1)
if(aps_extension_flag)	
while(more_rbsp_data())	
aps_extension_data_flag	u(1)
rbsp_trailing_bits()	
}	

[0148] These three types of APS parameters are discussed in turn below ALF APS

[0149] The ALF parameters are described in Adaptive loop filter data syntax elements (Table 5). First, four flags are dedicated to specify whether or not the ALF filters are transmitted for Luma and/or for Chroma and if the CC-ALF (Cross Component Adaptive Loop Filtering) is enabled for Cb component and Cr component. If the Luma filter flag is enabled, another flag is decoded to know if the clip values are signalled (alf_luma_clip_flag). Then the number of filters signalled is decoded using the alf_luma_num_filters_signalled_minus1 syntax element. If needed, the syntax element representing the ALF coefficients delta “alf_luma_coeff_delta_idx” is decoded for each enabled filter. Then absolute value and the sign for each coefficient of each filter are decoded.

[0150] If the alf_luma_clip_flag is enabled, the clip index for each coefficient of each enabled filter is decoded.

[0151] In the same way, the ALF chroma coefficients are decoded if needed.

[0152] If CC-ALF is enabled for Cr or Cb the number of filter are decoded (alf_cc_cb_filters_signalled_minus1 or alf_cc_cr_filters_signalled_minus1) and the related coefficients are decoded (alf_cc_cb_mapped_coeff_abs and alf_cc_cb_coeff_sign or respectively alf_cc_cr_mapped_coeff_abs and alf_cc_cr_coeff_sign)

TABLE 5

Adaptive loop filter data syntax	
	Descriptor
alf_data() {	
alf_luma_filter_signal_flag	u(1)
alf_chroma_filter_signal_flag	u(1)
alf_cc_cb_filter_signal_flag	u(1)
alf_cc_cr_filter_signal_flag	u(1)

TABLE 5-continued

Adaptive loop filter data syntax	
	Descriptor
if(alf_luma_filter_signal_flag) {	
alf_luma_clip_flag	u(1)
alf_luma_num_filters_signalled_minus1	ue(v)
if(alf_luma_num_filters_signalled_minus1 > 0)	
for(filtIdx = 0; filtIdx < NumAlfFilters; filtIdx++)	
alf_luma_coeff_delta_idx[filtIdx]	u(v)
for(sflIdx = 0; sflIdx <= alf_luma_num_filters_signalled_minus1; sflIdx++)	
for(j = 0; j < 12; j++) {	
alf_luma_coeff_abs[sflIdx][j]	ue(v)
if(alf_luma_coeff_abs[sflIdx][j])	
alf_luma_coeff_sign[sflIdx][j]	u(1)
}	
if(alf_luma_clip_flag)	
for(sflIdx = 0; sflIdx <= alf_luma_num_filters_signalled_minus1; sflIdx++)	
for(j = 0; j < 12; j++)	
alf_luma_clip_idx[sflIdx][j]	u(2)
}	
if(alf_chroma_filter_signal_flag) {	
alf_chroma_clip_flag	u(1)
alf_chroma_num_alt_filters_minus1	ue(v)
for(altIdx = 0; altIdx <= alf_chroma_num_alt_filters_minus1; altIdx++) {	
for(j = 0; j < 6; j++) {	
alf_chroma_coeff_abs[altIdx][j]	ue(v)
if(alf_chroma_coeff_abs[altIdx][j] > 0)	
alf_chroma_coeff_sign[altIdx][j]	u(1)
}	
if(alf_chroma_clip_flag)	
for(j = 0; j < 6; j++)	
alf_chroma_clip_idx[altIdx][j]	u(2)
}	
if(alf_cc_cb_filter_signal_flag) {	
alf_cc_cb_filters_signalled_minus1	ue(v)
for(k = 0; k < alf_cc_cb_filters_signalled_minus1 + 1; k++) {	
for(j = 0; j < 7; j++) {	
alf_cc_cb_mapped_coeff_abs[k][j]	u(3)
if(alf_cc_cb_mapped_coeff_abs[k][j])	
alf_cc_cb_coeff_sign[k][j]	u(1)
}	
}	
if(alf_cc_cr_filter_signal_flag) {	
alf_cc_cr_filters_signalled_minus1	ue(v)
for(k = 0; k < alf_cc_cr_filters_signalled_minus1 + 1; k++) {	
for(j = 0; j < 7; j++) {	
alf_cc_cr_mapped_coeff_abs[k][j]	u(3)
if(alf_cc_cr_mapped_coeff_abs[k][j])	
alf_cc_cr_coeff_sign[k][j]	u(1)
}	
}	
}	

Lmcs Syntax Elements for Both Luma Mapping and Chroma Scaling

[0153] The Table 6 below gives all the LMCS syntax elements which are coded in the adaptation parameter set (APS) syntax structure when the `aps_params_type` parameter is set to 1 (LMCS_APS). Up to four LMCS APS's can be used in a coded video sequence, however, only a single LMCS APS can be used for a given picture.

[0154] These parameters are used to build the forward and inverse mapping functions for Luma and the scaling function for Chroma.

TABLE 6

Luma mapping with chroma scaling data syntax	
	Descriptor
lmcs_data () {	
lmcs_min_bin_idx	ue(v)
lmcs_delta_max_bin_idx	ue(v)
lmcs_delta_cw_prec_minus1	ue(v)
for(i = lmcs_min_bin_idx; i <= LmcsMaxBinIdx; i++) {	
lmcs_delta_abs_cw[i]	u(v)
if(lmcs_delta_abs_cw[i] > 0)	

TABLE 6-continued

Luma mapping with chroma scaling data syntax	
	Descriptor
lms_delta_sign_cw_flag[i]	u(1)
}	
lms_delta_abs_crs	u(3)
if(lms_delta_abs_crs) > 0)	
lms_delta_sign_crs_flag	u(1)
}	

Scaling List APS

[0155] The scaling list offers the possibility to update the quantization matrix used for quantification. In VVC this scaling matrix is signalled in the APS as described in Scaling list data syntax elements (Table 7 Scaling list data syntax). The first syntax element specifies if the scaling matrix is used for the LFNST (Low Frequency Non-Separable Transform) tool based on the flag `scaling_matrix_for_lfnst_disabled_flag`. The second one is specified if the scaling list are used for Chroma components (`scaling_list_chroma_present_flag`). Then the syntax elements needed to build the scaling matrix are decoded (`scaling_list_copy_mode_flag`, `scaling_list_pred_id_delta`, `scaling_list_dc_coef`, `scaling_list_pred_mode_flag`, `scaling_list_delta_coef`).

TABLE 7

Scaling list data syntax	
	Descriptor
<code>scaling_list_data() {</code>	
<code>scaling_matrix_for_lfnst_disabled_flag</code>	u(1)
<code>scaling_list_chroma_present_flag</code>	u(1)
<code>for(id = 0; id < 28; id ++)</code>	
<code>matrixSize = (id < 2) ? 2 : ((id < 8) ? 4 : 8)</code>	
<code>if(scaling_list_chroma_present_flag (id % 3 == 2) (id == 27)) {</code>	
<code>scaling_list_copy_mode_flag[id]</code>	u(1)
<code>if(!scaling_list_copy_mode_flag[id])</code>	
<code>scaling_list_pred_mode_flag[id]</code>	u(1)
<code>if((scaling_list_copy_mode_flag[id] scaling_list_pred_mode_flag[id]) &&</code>	
<code>id != 0 && id != 2 && id != 8)</code>	
<code>scaling_list_pred_id_delta[id]</code>	ue(v)
<code>if(!scaling_list_copy_mode_flag[id]) {</code>	
<code>nextCoef = 0</code>	
<code>if(id > 13) {</code>	
<code>scaling_list_dc_coef[id - 14]</code>	se(v)
<code>nextCoef += scaling_list_dc_coef[id - 14]</code>	
<code>}</code>	
<code>for(i = 0; i < matrixSize * matrixSize; i++) {</code>	
<code>x = DiagScanOrder[3][3][i][0]</code>	
<code>y = DiagScanOrder[3][3][i][1]</code>	
<code>if(!(id > 25 && x >= 4 && y >= 4)) {</code>	
<code>scaling_list_delta_coef[id][i]</code>	se(v)
<code>nextCoef += scaling_list_delta_coef[id][i]</code>	
<code>}</code>	
<code>ScalingList[id][i] = nextCoef</code>	
<code>}</code>	
<code>}</code>	
<code>}</code>	
<code>}</code>	
<code>}</code>	

Picture Header

[0156] The picture header is transmitted at the beginning of each picture before the other Slice Data. This is very large compared to the previous headers in the previous drafts of the standard. A complete description of all these parameters

can be found in JVET-Q2001-vD. Table 9 shows these parameters in the current picture header decoding syntax.

[0157] The related syntax elements which can be decoded are related to:

- [0158]** the usage of this picture, reference frame or not
- [0159]** The type of picture
- [0160]** output frame.
- [0161]** The number of the Picture
- [0162]** subpicture usage if needed
- [0163]** reference picture lists if needed
- [0164]** colour plane if needed
- [0165]** partitioning update if overriding flag is enabled
- [0166]** delta QP parameters if needed
- [0167]** Motion information parameters if needed
- [0168]** ALF parameters if needed
- [0169]** SAO parameters if needed
- [0170]** quantification parameters if needed
- [0171]** LMCS parameters if needed
- [0172]** Scaling list parameters if needed
- [0173]** picture header extension if needed
- [0174]** Etc. . . .

Picture “Type”

[0175] The first flag is the `gdr_or_irap_pic_flag` which indicates if the current picture is a resynchronisation picture

(IRAP or GDR). If this flag is true, the `gdr_pic_flag` is decoded to know if the current picture is an IRAP or a GDR picture.

[0176] Then the `ph_inter_slice_allowed_flag` is decoded to identify that the Inter slice is allowed.

[0177] When they are allowed, the flag `ph_intra_slice_allowed_flag` is decoded to know if the Intra slice are allowed for the current picture.

[0178] Then the `non_reference_picture_flag`, the `ph_pic_parameter_set_id` indicating the PPS ID and the picture order count `ph_pic_order_cnt_lsb` are decoded. The picture order count gives the number of the current picture.

[0179] If the picture is a GDR or an IRAP picture, the flag `no_output_of_prior_pics_flag` is decoded.

[0180] And if the picture is a GDR the recovery `poc_cnt` is decoded. Then `ph_poc_msb_present_flag` and `poc_msb_val` are decoded if needed.

ALF

[0181] After these parameters describing important information on the current picture, the set of ALF APS id syntax elements are decoded if ALF is enabled at SPS level and if ALF is enabled at picture header level. ALF is enabled at SPS level thanks to the `sps_alf_enabled_flag` flag. And ALF signalling is enabled at picture header level thanks to the `alf_info_in_ph_flag` equal to 1 otherwise (`alf_info_in_ph_flag` equal to 0) ALF is signalled at slice level. The `alf_info_in_ph_flag` is defined as the following:

[0182] “`alf_info_in_ph_flag` equal to 1 specifies that ALF information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. `alf_info_in_ph_flag` equal to 0 specifies that ALF information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure.”

[0183] First the `ph_alf_enabled_present_flag` is decoded to determine whether or not if the `ph_alf_enabled_flag` should be decoded. If the `ph_alf_enabled_flag` is enabled, ALF is enabled for all slices of the current picture.

[0184] If ALF is enabled, the amount of ALF APS id for luma is decoded using the `pic_num_alf_aps_ids_luma` syntax element. For each APS id, the APS id value for luma is decoded “`ph_alf_aps_id_luma`”.

[0185] For chroma the syntax element, `ph_alf_chroma_idc` is decoded to determine whether or not ALF is enabled for Chroma, for Cr only, or for Cb only. If it is enabled, the value of the APS ID for Chroma is decoded using the `ph_alf_aps_id_chroma` syntax element.

[0186] In the way the APS ID for CC-ALF method are decoded if needed for Cb and/or CR components

LMCS

[0187] The set of LMCS APS ID syntax elements is then decoded if LMCS was enabled at SPS level. First the `ph_lmcs_enabled_flag` is decoded to determine whether or not LMCS is enabled for the current picture. If LMCS is enabled, the ID value is decoded `ph_lmcs_aps_id`. For Chroma only the `ph_chroma_residual_scale_flag` is decoded to enable or disable the method for Chroma.

Scaling List

[0188] The set of scaling list APS ID is then decoded if the scaling list is enabled at SPS level. The `ph_scaling_list_present_flag` is decoded to determine whether or not the scaling matrix is enabled for the current picture. And the value of the APS ID, `ph_scaling_list_aps_id`, is then decoded.

Subpicture

[0189] The Subpicture parameters are enabled when they are enabled at SPS and if the subpicture id signalling is disabled. It also contains some information on virtual boundaries. For the sub picture parameters eight syntax elements are defined:

- [0190] `ph_virtual_boundaries_present_flag`
- [0191] `ph_num_ver_virtual_boundaries`
- [0192] `ph_virtual_boundaries_pos_x[i]`
- [0193] `ph_num_hor_virtual_boundaries`
- [0194] `ph_virtual_boundaries_pos_y[i]`

Output Flag

[0195] These subpicture parameters are followed by the `pic_output_flag` if present.

Reference Picture Lists

[0196] If the reference picture lists are signalled in the picture header (thanks to `rpl_info_in_ph_flag` equal to 1), then the parameters for the reference picture lists are decoded `ref_pic_lists()` it contains the following syntax elements:

- [0197] `rpl_sps_flag[ ]`
- [0198] `rpl_idx[ ]`
- [0199] `poc_lsb_ltr[ ][ ]`
- [0200] `delta_poc_msb_present_flag[ ][ ]`
- [0201] `delta_poc_msb_cycle_ltr[ ][ ]`

Partitioning

[0202] The set of partitioning parameters is decoded if needed and contains the following syntax elements:

- [0203] `partition_constraints_override_flag`
- [0204] `ph_log 2_diff_min_qt_min_cb_intra_slice_luma`
- [0205] `ph_max_mtt_hierarchy_depth_intra_slice_luma`
- [0206] `ph_log 2_diff_max_bt_min_at_intra_slice_luma`
- [0207] `ph_log 2_diff_max_tt_min_at_intra_slice_luma`
- [0208] `ph_log 2_diff_min_qt_min_cb_intra_slice_chroma`
- [0209] `ph_max_mtt_hierarchy_depth_intra_slice_chroma`
- [0210] `ph_log 2_diff_max_bt_min_qt_intra_slice_chroma`
- [0211] `ph_log 2_diff_max_tt_min_at_intra_slice_chroma`
- [0212] `ph_log 2_diff_min_qt_min_cb_inter_slice`
- [0213] `ph_max_mtt_hierarchy_depth_inter_slice`
- [0214] `ph_log 2_diff_max_bt_min_at_inter_slice`
- [0215] `ph_log 2_diff_max_tt_min_at_inter_slice`

Weighted Prediction

[0216] The weighted prediction parameters `pred_weight_table()` are decoded if the weighted prediction method is enabled at PPS level and if the weighted prediction parameters are signalled in the picture header (`wp_info_in_ph_flag` equal to 1).

[0217] The `pred_weight_table()` contains the weighted prediction parameters for List L0 and for list L1 when bi-prediction weighted prediction is enabled. When the weighted prediction parameters are transmitted in the picture

header the number of weights for each list are explicitly transmitted as depicted in the `pred_weight_table( )` syntax table Table 8.

TABLE 8

Weighted prediction parameters syntax	
	Descriptor
<code>pred_weight_table() {</code>	
<code>luma_log2_weight_denom</code>	<code>ue(v)</code>
<code>if(ChromaArrayType != 0)</code>	
<code>delta_chroma_log2_weight_denom</code>	<code>se(v)</code>
<code>if(wp_info_in_ph_flag)</code>	
<code>num_l0_weights</code>	<code>ue(v)</code>
<code>for(i = 0; i < NumWeightsL0; i++)</code>	
<code>luma_weight_l0_flag[i]</code>	<code>u(1)</code>
<code>if(ChromaArrayType != 0)</code>	
<code>for(i = 0; i < NumWeightsL0; i++)</code>	
<code>chroma_weight_l0_flag[i]</code>	<code>u(1)</code>
<code>for(i = 0; i < NumWeightsL0; i++) {</code>	
<code>if(luma_weight_l0_flag[i]) {</code>	
<code>delta_luma_weight_l0[i]</code>	<code>se(v)</code>
<code>luma_offset_l0[i]</code>	<code>se(v)</code>
<code>}</code>	
<code>if(chroma_weight_l0_flag[i])</code>	
<code>for(j = 0; j < 2; j++) {</code>	
<code>delta_chroma_weight_l0[i][j]</code>	<code>se(v)</code>
<code>delta_chroma_offset_l0[i][j]</code>	<code>se(v)</code>
<code>}</code>	
<code>}</code>	
<code>if(pps_weighted_bipred_flag && wp_info_in_ph_flag)</code>	<code>ue(v)</code>
<code>num_l1_weights</code>	
<code>for(i = 0; i < NumWeightsL1; i++)</code>	
<code>luma_weight_l1_flag[i]</code>	<code>u(1)</code>

TABLE 8-continued

Weighted prediction parameters syntax	
	Descriptor
<code>if(ChromaArrayType != 0)</code>	
<code>for(i = 0; i < NumWeightsL1; i++)</code>	
<code>chroma_weight_l1_flag[i]</code>	<code>u(1)</code>
<code>for(i = 0; i < NumWeightsL1; i++) {</code>	
<code>if(luma_weight_l1_flag[i]) {</code>	
<code>delta_luma_weight_l1[i]</code>	<code>se(v)</code>
<code>luma_offset_l1[i]</code>	<code>se(v)</code>
<code>}</code>	
<code>if(chroma_weight_l1_flag[i])</code>	
<code>for(j = 0; j < 2; j++) {</code>	
<code>delta_chroma_weight_l1[i][j]</code>	<code>se(v)</code>
<code>delta_chroma_offset_l1[i][j]</code>	<code>se(v)</code>
<code>}</code>	
<code>}</code>	
<code>}</code>	

Delta QP

[0218] When the picture is Intra the `ph_cu_qp_delta_subdiv_intra_slice` and the `ph_cu_chroma_qp_offset_subdiv_intra_slice` are decoded if needed. And if Inter slice is allowed the `ph_cu_qp_delta_subdiv_inter_slice` and the `ph_cu_chroma_qp_offset_subdiv_inter_slice` are decoded if needed. Finally, the picture header extension syntax elements are decoded if needed.

[0219] All parameters `alf_info_in_ph_flag`, `rpl_info_in_ph_flag`, `qp_delta_info_in_ph_flag`, `sao_info_in_ph_flag`, `dbf_info_in_ph_flag`, `wp_info_in_ph_flag` are signalled in the PPS.

TABLE 9

Picture header structure	
	Descriptor
<code>picture_header_structure() {</code>	
<code>gdr_or_irap_pic_flag</code>	<code>u(1)</code>
<code>if(gdr_or_irap_pic_flag)</code>	
<code>gdr_pic_flag</code>	<code>u(1)</code>
<code>ph_inter_slice_allowed_flag</code>	<code>u(1)</code>
<code>if(ph_inter_slice_allowed_flag)</code>	
<code>ph_intra_slice_allowed_flag</code>	<code>u(1)</code>
<code>non_reference_picture_flag</code>	<code>u(1)</code>
<code>ph_pic_parameter_set_id</code>	<code>ue(v)</code>
<code>ph_pic_order_cnt_lsb</code>	<code>u(v)</code>
<code>if(gdr_or_irap_pic_flag)</code>	
<code>no_output_of_prior_pics_flag</code>	<code>u(1)</code>
<code>if(gdr_pic_flag)</code>	
<code>recovery_poc_cnt</code>	<code>ue(v)</code>
<code>for(i = 0; i < NumExtraPhBits; i++)</code>	
<code>ph_extra_bit[i]</code>	<code>u(1)</code>
<code>if(sps_poc_msb_flag) {</code>	
<code>ph_poc_msb_present_flag</code>	<code>u(1)</code>
<code>if(ph_poc_msb_present_flag)</code>	
<code>poc_msb_val</code>	<code>u(v)</code>
<code>}</code>	
<code>if(sps_alf_enabled_flag && alf_info_in_ph_flag) {</code>	
<code>ph_alf_enabled_flag</code>	<code>u(1)</code>
<code>if(ph_alf_enabled_flag) {</code>	
<code>ph_num_alf_aps_ids_luma</code>	<code>u(3)</code>
<code>for(i = 0; i < ph_num_alf_aps_ids_luma; i++)</code>	
<code>ph_alf_aps_id_luma[i]</code>	<code>u(3)</code>
<code>if(ChromaArrayType != 0)</code>	
<code>ph_alf_chroma_idc</code>	<code>u(2)</code>
<code>if(ph_alf_chroma_idc > 0)</code>	
<code>ph_alf_aps_id_chroma</code>	<code>u(3)</code>
<code>if(sps_ccalf_enabled_flag) {</code>	
<code>ph_cc_alf_cb_enabled_flag</code>	<code>u(1)</code>

TABLE 9-continued

Picture header structure		Descriptor
if(ph_cc_alf_cb_enabled_flag)		
ph_cc_alf_cb_aps_id		u(3)
ph_cc_alf_cr_enabled_flag		u(1)
if(ph_cc_alf_cr_enabled_flag)		
ph_cc_alf_cr_aps_id		u(3)
}		
}		
if(sps_lmcs_enabled_flag) {		
ph_lmcs_enabled_flag		u(1)
if(ph_lmcs_enabled_flag) {		
ph_lmcs_aps_id		u(2)
if(ChromaArrayType != 0)		
ph_chroma_residual_scale_flag		u(1)
}		
}		
if(sps_scaling_list_enabled_flag) {		
ph_scaling_list_present_flag		u(1)
if(ph_scaling_list_present_flag)		
ph_scaling_list_aps_id		u(3)
}		
if(sps_virtual_boundaries_enabled_flag && !sps_virtual_boundaries_present_flag) {		
ph_virtual_boundaries_present_flag		u(1)
if(ph_virtual_boundaries_present_flag) {		
ph_num_ver_virtual_boundaries		u(2)
for(i = 0; i < ph_num_ver_virtual_boundaries; i++)		
ph_virtual_boundaries_pos_x[i]		u(13)
ph_num_hor_virtual_boundaries		u(2)
for(i = 0; i < ph_num_hor_virtual_boundaries; i++)		
ph_virtual_boundaries_pos_y[i]		u(13)
}		
}		
if(output_flag_present_flag)		
pic_output_flag		u(1)
if(rpl_info_in_ph_flag)		
ref_pic_lists()		
if(partition_constraints_override_enabled_flag)		
partition_constraints_override_flag		u(1)
if(ph_intra_slice_allowed_flag) {		
if(partition_constraints_override_flag) {		
ph_log2_diff_min_qt_min_cb_intra_slice_luma		ue(v)
ph_max_mtt_hierarchy_depth_intra_slice_luma		ue(v)
if(ph_max_mtt_hierarchy_depth_intra_slice_luma != 0) {		
ph_log2_diff_max_bt_min_qt_intra_slice_luma		ue(v)
ph_log2_diff_max_tt_min_qt_intra_slice_luma		ue(v)
}		
if(qtbtt_dual_tree_intra_flag) {		
ph_log2_diff_min_qt_min_cb_intra_slice_chroma		ue(v)
ph_max_mtt_hierarchy_depth_intra_slice_chroma		ue(v)
if(ph_max_mtt_hierarchy_depth_intra_slice_chroma != 0) {		
ph_log2_diff_max_bt_min_qt_intra_slice_chroma		ue(v)
ph_log2_diff_max_tt_min_qt_intra_slice_chroma		ue(v)
}		
}		
}		
if(cu_qp_delta_enabled_flag)		
ph_cu_qp_delta_subdiv_intra_slice		ue(v)
if(pps_cu_chroma_qp_offset_list_enabled_flag)		
ph_cu_chroma_qp_offset_subdiv_intra_slice		ue(v)
}		
if(ph_inter_slice_allowed_flag) {		
if(partition_constraints_override_flag) {		
ph_log2_diff_min_qt_min_cb_inter_slice		ue(v)
ph_max_mtt_hierarchy_depth_inter_slice		ue(v)
if(ph_max_mtt_hierarchy_depth_inter_slice != 0) {		
ph_log2_diff_max_bt_min_qt_inter_slice		ue(v)
ph_log2_diff_max_tt_min_qt_inter_slice		ue(v)
}		
}		
if(cu_qp_delta_enabled_flag)		
ph_cu_qp_delta_subdiv_inter_slice		ue(v)
if(pps_cu_chroma_qp_offset_list_enabled_flag)		
ph_cu_chroma_qp_offset_subdiv_inter_slice		ue(v)

TABLE 9-continued

Picture header structure	
	Descriptor
if(sps_temporal_mvp_enabled_flag) {	
ph_temporal_mvp_enabled_flag	u(1)
if(ph_temporal_mvp_enabled_flag && rpl_info_in_ph_flag) {	
ph_collocated_from_l0_flag	u(1)
if((ph_collocated_from_l0_flag &&	
num_ref_entries[0][RplIdx[0]] > 1)	
(!ph_collocated_from_l0_flag &&	
num_ref_entries[1][RplIdx[1]] > 1))	
ph_collocated_ref_idx	ue(v)
}	
}	
mvd_l1_zero_flag	u(1)
if(sps_fpel_mmvd_enabled_flag)	
ph_fpel_mmvd_enabled_flag	u(1)
if(sps_bdof_pic_present_flag)	
ph_disable_bdof_flag	u(1)
if(sps_dmvr_pic_present_flag)	
ph_disable_dmvr_flag	u(1)
if(sps_prof_pic_present_flag)	
ph_disable_prof_flag	u(1)
if((pps_weighted_pred_flag pps_weighted_bipred_flag) && wp_info_in_ph_flag)	
pred_weight_table()	
}	
if(qp_delta_info_in_ph_flag)	
ph_qp_delta	se(v)
if(sps_joint_cbr_enabled_flag)	
ph_joint_cbr_sign_flag	u(1)
if(sps_sao_enabled_flag && sao_info_in_ph_flag) {	
ph_sao_luma_enabled_flag	u(1)
if(ChromaArrayType != 0)	
ph_sao_chroma_enabled_flag	u(1)
}	
if(sps_dep_quant_enabled_flag)	
ph_dep_quant_enabled_flag	u(1)
if(sps_sign_data_hiding_enabled_flag && !ph_dep_quant_enabled_flag)	
pic_sign_data_hiding_enabled_flag	u(1)
if(deblocking_filter_override_enabled_flag && dbf_info_in_ph_flag) {	
ph_deblocking_filter_override_flag	u(1)
if(ph_deblocking_filter_override_flag) {	
ph_deblocking_filter_disabled_flag	u(1)
if(!ph_deblocking_filter_disabled_flag) {	
ph_beta_offset_div2	se(v)
ph_tc_offset_div2	se(v)
ph_cb_beta_offset_div2	se(v)
ph_cb_tc_offset_div2	se(v)
ph_cr_beta_offset_div2	se(v)
ph_cr_tc_offset_div2	se(v)
}	
}	
}	
}	
if(picture_header_extension_present_flag) {	
ph_extension_length	ue(v)
for(i = 0; i < ph_extension_length; i++)	
ph_extension_data_byte[i]	u(8)
}	
}	

Slice Header

[0220] The Slice header is transmitted at the beginning of each slice. The slice header contains about 65 syntax elements. This is very large compared to the previous slice

header in earlier video coding standards. A complete description of all the slice header parameters can be found in JVET-Q2001-vD. Table 10 shows these parameters in a current slice header decoding syntax.

TABLE 10

Partial Slice header	
	Descriptor
slice_header() {	
picture_header_in_slice_header_flag	u(1)
if(picture_header_in_slice_header_flag)	
picture_header_structure()	
if(subpic_info_present_flag)	
slice_subpic_id	u(v)
if((rect_slice_flag && NumSlicesInSubpic[CurrSubpicIdx] > 1)	
(!rect_slice_flag && NumTilesInPic > 1))	
slice_address	u(v)
for(i = 0; i < NumExtraShBits; i++)	
sh_extra_bit[i]	u(1)
if(!rect_slice_flag && NumTilesInPic > 1)	
num_tiles_in_slice_minus1	ue(v)
if(ph_inter_slice_allowed_flag)	
slice_type	ue(v)
if(sps_alf_enabled_flag && !alf_info_in_ph_flag) {	
slice_alf_enabled_flag	u(1)
if(slice_alf_enabled_flag) {	
slice_num_alf_aps_ids_luma	u(3)
for(i = 0; i < slice_num_alf_aps_ids_luma; i++)	
slice_alf_aps_id_luma[i]	u(3)
if(ChromaArrayType != 0)	
slice_alf_chroma_idc	u(2)
if(slice_alf_chroma_idc)	
slice_alf_aps_id_chroma	u(3)
if(sps_ccalf_enabled_flag) {	
slice_cc_alf_cb_enabled_flag	u(1)
if(slice_cc_alf_cb_enabled_flag)	
slice_cc_alf_cb_aps_id	u(3)
slice_cc_alf_cr_enabled_flag	u(1)
if(slice_cc_alf_cr_enabled_flag)	
slice_cc_alf_cr_aps_id	u(3)
}	
}	
if(separate_colour_plane_flag == 1)	
colour_plane_id	u(2)
if(!rpl_info_in_ph_flag && ((nal_unit_type != IDR_W_RADL && nal_unit_type !=	
IDR_N_LP) sps_jdr_rpl_present_flag))	
ref_pic_lists()	
if((rpl_info_in_ph_flag ((nal_unit_type != IDR_W_RADL && nal_unit_type !=	
IDR_N_LP) sps_jdr_rpl_present_flag)) &&	
((slice_type != I && num_ref_entries[0][RplIdx[0]] > 1)	
(slice_type == B && num_ref_entries[1][RplIdx[1]] > 1)) {	
num_ref_idx_active_override_flag	u(1)
if(num_ref_idx_active_override_flag)	
for(i = 0; i < (slice_type == B ? 2 : 1); i++)	
if(num_ref_entries[i][RplIdx[i]] > 1)	
num_ref_idx_active_minus1[i]	ue(v)
}	
if(slice_type != I) {	
if(cabac_init_present_flag)	
cabac_init_flag	u(1)
if(ph_temporal_mvp_enabled_flag && !rpl_info_in_ph_flag) {	
if(slice_type == B)	
slice_collocated_from_l0_flag	u(1)
if((slice_collocated_from_l0_flag && NumRefIdxActive[0] > 1)	
(! slice_collocated_from_l0_flag && NumRefIdxActive[1] > 1))	
slice_collocated_ref_idx	ue(v)
}	
if(!wp_info_in_ph_flag && ((pps_weighted_pred_flag && slice_type == P)	
(pps_weighted_bipred_flag && slice_type == B)))	
pred_weight_table()	
}	
if(!qp_delta_info_in_ph_flag)	
slice_qp_delta	se(v)

TABLE 10-continued

Partial Slice header	
	Descriptor
if(pps_slice_chroma_qp_offsets_present_flag) {	
slice_cb_qp_offset	se(v)
slice_cr_qp_offset	se(v)
if(sps_joint_cbr_enabled_flag)	
slice_joint_cbr_qp_offset	se(v)
}	
if(pps_cu_chroma_qp_offset_list_enabled_flag)	
cu_chroma_qp_offset_enabled_flag	u(1)
if(sps_sao_enabled_flag && !sao_info_in_ph_flag) {	
slice_sao_luma_flag	u(1)
if(ChromaArrayType != 0)	
slice_sao_chroma_flag	u(1)
}	
if(deblocking_filter_override_enabled_flag && !dbf_info_in_ph_flag)	
slice_deblocking_filter_override_flag	u(1)
if(slice_deblocking_filter_override_flag) {	
slice_deblocking_filter_disabled_flag	u(1)
if(!slice_deblocking_filter_disabled_flag) {	
slice_beta_offset_div2	se(v)
slice_tc_offset_div2	se(v)
slice_cb_beta_offset_div2	se(v)
slice_cb_tc_offset_div2	se(v)
slice_cr_beta_offset_div2	se(v)
slice_cr_tc_offset_div2	se(v)
}	
}	
slice_ts_residual_coding_disabled_flag	u(1)
if(ph_lmcs_enabled_flag)	
slice_lmcs_enabled_flag	u(1)
if(ph_scaling_list_present_flag)	
slice_scaling_list_present_flag	u(1)
if(NumEntryPoints > 0) {	
offset_len_minus1	ue(v)
for(i = 0; i < NumEntryPoints; i++)	
entry_point_offset_minus1[i]	u(v)
}	
if(slice_header_extension_present_flag) {	
slice_header_extension_length	ue(v)
for(i = 0; i < slice_header_extension_length; i++)	
slice_header_extension_data_byte[i]	u(8)
}	
byte_alignment()	
}	

[0221] First the `picture_header_in_slice_header_flag` is decoded to know if the `picture_header_structure()` is present in the slice header.

[0222] The `slice_subpic_id` if needed, is then decoded to determine the sub picture id of the current slice. Then the `slice_address` is decoded to determine the address of the current slice. The `num_tiles_in_slice_minus1` is then decoded if the number of tiles in the current picture is greater than one.

[0223] Then the `slice_type` is decoded.

[0224] If ALF is enabled at SPS level (`sps_alf_enabled_flag`) and if ALF is signalled in the slice header (`alf_info_in_ph_flag` equal to 0), then ALF information is decoded. This includes a flag indicating that ALF is enabled for the current slice (`slice_alf_enabled_flag`). If it is enabled, the number of APS ALF ID for luma (`slice_num_alf_aps_ids_luma`) is decoded, then the APS ID are decoded (`slice_alf_aps_id_luma[i]`). Then the `slice_alf_chroma_idc` is decoded to know if ALF is enabled for the Chroma components and which chroma component it is enabled. Then the APS ID for Chroma is decoded `slice_alf_aps_id_chroma` if needed. In the same way, the `slice_cc_alf_cb_enabled_flag` is decoded,

if needed, to know if the CC ALF method is enabled. If CC ALF is enabled, the related APS ID for CR and/or CB are decoded if CC ALF is enabled for CR and/or CB.

[0225] If the colour planes are transmitted independently (`separate_colour_plane_flag` equals to 1) the `colour_plane_id` is decoded.

[0226] When the reference picture lists are not transmitted in the picture header (`rpl_info_in_ph_flag` equal to 0) and when the Nal unit is not an IDR or if the reference pictures lists are transmitted for IDR pictures (`sps_idr_rpl_present_flag` equals to 1) then the Reference picture lists parameters are decoded; these are similar to those in the picture header.

[0227] If the reference picture lists are transmitted in the picture header (`rpl_info_in_ph_flag` equal to 1) or the Nal unit is not an IDR or if the reference picture lists are transmitted for IDR pictures (`sps_idr_rpl_present_flag` equals to 1) and if the number of reference for at least one list is superior to 1, the override flag `num_ref_idx_active_override_flag` is decoded. If this flag is enabled the reference index for each list are decoded.

[0228] When the slice type is not intra and if needed the `cabac_init_flag` is decoded. If the reference picture lists are

transmitted in the slice header and come other conditions, the slice_collocated_from_l0_flag and the slice_collocated_ref_idx are decoded. These data are related to the CABAC coding and the motion vector collocated.

[0229] In the same way, when the slice type is not Intra, the parameters of the weighted prediction pred_weight_table() are decoded.

[0230] The slice_qp_delta is decoded bif the delta QP information is transmitted in the slice header (qp_delta_info_in_ph_flag equal to 0). If needed the syntax elements, slice_cb_qp_offset, slice_cr_qp_offset, slice_joint_cbr_qp_offset, cu_chroma_qp_offset_enabled_flag are decoded.

[0231] If the SAO information are transmitted in the slice header (sao_info_in_ph_flag equal to 0) and if it is enabled at SPS level (sps_sao_enabled_flag), the enabled flags for SAO are decoded for both luma and chroma: slice_sao_luma_flag, slice_sao_chroma_flag. Then the deblocking filter parameters are decoded if they are signalled in the slice header (dbf_info_in_ph_flag equal to 0).

[0232] The flag slice_ts_residual_coding_disabled_flag is systematically decoded to know if the Transform Skip residual coding method is enabled for the current slice.

[0233] If LMCS was enabled in the picture header (ph_lmcs_enabled_flag equal 1), the flag slice_lmcs_enabled_flag is decoded.

[0234] In the same way, if the scaling list was enabled in the picture header (phpic_scaling_list_presentenabled_flag equal 1), the flag slice_scaling_list_present_flag is decoded.

[0235] Then other parameters are decoded if needed.

Picture Header in the Slice Header

[0236] In a particular signalling way, the picture header 708 can be signalled inside the slice header 710 as depicted in the FIG. 7. In that case there is no Nal unit containing only the picture header 608. The units 701, 702, 703, 704, 705, 706, 707, 720, and 740 correspond to 601, 602, 603, 604, 605, 606, 620, and 640 of FIG. 6 and, thus can be understood from the preceding description. This can be enabled in the slice header thanks to the flag picture_header_in_slice_header_flag. Moreover, when the picture header is

present in the slice header, picture_header_in_slice_header_flag equal to 0 specifies that the PH syntax structure is not present in the slice header.

[0238] It is a requirement of f bitstream conformance that the value of picture_header_in_slice_header_flag shall be the same in all coded slices in a CLVS. When picture_header_in_slice_header_flag is equal to 1 for a coded slice, it is a requirement of bitstream conformance that no VCL NAL unit with nal_unit_type equal to PH_NUT shall be present in the CLVS.

[0239] When picture_header_in_slice_header_flag is equal to 0, all coded slices in the current picture shall have picture_header_in_slice_header_flag is equal to 0, and the current PU shall have a PH NAL unit.

[0240] The picture_header_structure() contains syntax elements of the picture_rbsp() except the stuffing bits rbsp_trailing_bits()

[0241] Interaction between Picture Header in Slice Header and tools signalling in Picture Header and Slice Header

[0242] The QP Delta information, the reference picture lists parameters, the deblocking filter parameters, the sample adaptive offsets parameters, the weighted prediction parameters and the ALF parameters can be transmitted in the picture header or in the slice header thanks to the respective flags:

[0243] qp_delta_info_in_ph_flag

[0244] rpl_info_in_ph_flag

[0245] dbf_info_in_ph_flag

[0246] sao_info_in_ph_flag

[0247] wp_info_in_ph_flag

[0248] alf_info_in_ph_flag

[0249] These flags are transmitted in the PPS.

[0250] As shown in Table 11 Summary of XXX signalling according to the flags picture_header_in_slice_header_flag and xxx_info_in_ph_flag, by considering “xxx” for one of the mentioned tools, xxx can be signalled in the slice header when xxx_info_in_ph_flag is set equal to 0 with the current syntax. In other words, we use the shorthand XXX or xxx in the subsequent description to refer to the information types (examples of which are given above) that may be signalled in both the picture header and the slice header.

TABLE 11

Summary of XXX signalling according to the flags picture_header_in_slice_header_flag and xxx_info_in_ph_flag		
xxx_info_in_ph_flag = 1 xxx_info_in_ph_flag = 0		
picture_header_in_slice_header_flag = 1	XXX in Picture Header	XXX in Slice Header or not present
picture_header_in_slice_header_flag = 0	XXX in Picture Header	XXX in Slice Header or not present

signalled inside the slice header, the picture shall contain only one slice. So, there is always only one picture header per picture. Moreover, the flag picture_header_in_slice_header_flag shall have the same value for all pictures of a CLVS (Coded Layer Video Sequence). It means that all pictures between two IRAP including the first IRAP has only one slice per picture.

[0237] The flag picture_header_in_slice_header_flag is defined as the following: “picture_header_in_slice_header_flag equal to 1 specifies that the PH syntax structure is

[0251] When the picture header is in the slice header this implies only one slice is present for the current picture. Thus, having the information transmitted or transmittable for both slice and picture doesn’t increase the flexibility for encoder or decoder as the parameters will be the same. In other words, if the information is in the picture header, corresponding information in the slice header will be redundant. Similarly, if the information is in the slice header, corresponding information in the picture header will be redundant. Embodiments described herein simplify the decoder implementation by limiting the redundancies in the

signaling of the same coding. In particular, in embodiments, the information is permitted to be in the picture header but not the slice header when the picture header is signalled in the slice header.

Streaming Applications

[0252] Some streaming applications only extract certain parts of the bitstream. These extractions can be spatial (as the sub-picture) or temporal (a subpart of the video sequence). Then these extracted parts can be merged with other bitstreams. Some other reduce the frame rate by extracting only some frames. Generally, the main aim of these streaming applications is to use the maximum of the allowed bandwidth to produce the maximum quality to the end user.

[0253] In VVC, the APS ID numbering has been limited for frame rate reduction, in order that a new APS id number for a frame can't be used for a frame at an upper level in the temporal hierarchy. However, for streaming applications which extract parts of the bitstream the APS ID needs to be tracked to determine which APS should be kept for a sub part of the bitstream as the frame (as IRAP) don't reset the numbering of the APS ID.

LMCS (Luma Mapping with Chroma Scaling)

[0254] The Luma Mapping with Chroma scaling (LMCS) technique is a sample value conversion method applied on a block before applying the loop filters in a video decoder like VVC.

[0255] The LMCS can be divided into two sub-tools. The first one is applied on Luma block while the second sub-tool is applied on Chroma blocks as described below:

[0256] 1) The first sub-tool is an in-loop mapping of the Luma component based on adaptive piecewise linear models. The in-loop mapping of the Luma component adjusts the dynamic range of the input signal by redistributing the codewords across the dynamic range to improve compression efficiency. Luma mapping makes use of a forward mapping function into the "mapped domain" and a corresponding inverse mapping function to come back in the "input domain".

[0257] 2) The second sub-tool is related to the chroma components where a luma-dependent chroma residual scaling is applied. Chroma residual scaling is designed to compensate for the interaction between the luma signal and its corresponding chroma signals. Chroma residual scaling depends on the average value of top and/or left reconstructed neighbouring luma samples of the current block.

[0258] Like most other tools in video coder like VVC, LMCS can be enabled/disabled at the sequence level using an SPS flag. Whether chroma residual scaling is enabled or not is also signalled at the slice level. If luma mapping is enabled, an additional flag is signalled to indicate if luma-dependent chroma residual scaling is enabled or not. When luma mapping is not used, luma-dependent chroma residual scaling is fully disabled. In addition, luma-dependent chroma residual scaling is always disabled for the chroma blocks whose size is less than or equal to 4.

[0259] FIG. 8 shows the principle of the LMCS as explained above for the Luma mapping sub-tool. The hatched blocks in FIG. 8 are the new LMCS functional blocks, including forward and inverse mapping of the luma signal. It is important to note that, when using LMCS, some decoding operations are applied in the "mapped domain".

These operations are represented by blocks in dashed lines in this FIG. 8. They typically correspond to the inverse quantization, the inverse transform, the luma intra prediction and the reconstruction step which consists in adding the luma prediction with the luma residual. Conversely, the solid line blocks in FIG. 8 indicate where the decoding process is applied in the original (i.e., non-mapped) domain and this includes the loop filtering such as deblocking, ALF, and SAO, the motion compensated prediction, and the storage of decoded pictures as reference pictures (DPB).

[0260] FIG. 9 shows a similar diagram as FIG. 8 but this time this is for the Chroma scaling sub-tool of the LMCS tool. The hatched block in FIG. 9 is the new LMCS functional block which includes the luma-dependent chroma scaling process. However, in Chroma, there are some important differences compared to the Luma case. Here only the inverse quantization and the inverse transform represented by block in dash lines are performed in the "mapped domain" for the Chroma samples. All the other steps of Intra Chroma prediction, motion compensation, loop filtering are performed in the original domain. As depicted in FIG. 9, there is only a scaling process and there is no forward and inverse processing as for the Luma mapping.

Luma Mapping by Using Piece Wise Linear Model.

[0261] The luma mapping sub-tool is using a piecewise linear model. It means that the piecewise linear model separates the input signal dynamic range into 16 equal sub-ranges, and for each sub-range, its linear mapping parameters are expressed using the number of codewords assigned to that range.

Semantics for Luma Mapping

[0262] The syntax element `lmcs_min_bin_idx` specifies the minimum bin index used in the luma mapping with chroma scaling (LMCS) construction process. The value of `lmcs_min_bin_idx` shall be in the range of 0 to 15, inclusive.

[0263] The syntax element `lmcs_delta_max_bin_idx` specifies the delta value between 15 and the maximum bin index `LmcsMaxBinIdx` used in the luma mapping with chroma scaling construction process. The value of `lmcs_delta_max_bin_idx` shall be in the range of 0 to 15, inclusive. The value of `LmcsMaxBinIdx` is set equal to `15-lmcs_delta_max_bin_idx`. The value of `LmcsMaxBinIdx` shall be greater than or equal to `lmcs_min_bin_idx`.

[0264] The syntax element `lmcs_delta_cw_prec_minus1` plus 1 specifies the number of bits used for the representation of the syntax `lmcs_delta_abs_cw[i]`.

[0265] The syntax element `lmcs_delta_abs_cw[i]` specifies the absolute delta codeword value for the i_{th} bin.

[0266] The syntax element `lmcs_delta_sign_cw_flag[i]` specifies the sign of the variable `lmcsDeltaCW[i]`. When `lmcs_delta_sign_cw_flag[i]` is not present, it is inferred to be equal to 0.

Lmcs Intermediate Variables Computation for Luma Mapping

[0267] In order to apply the forward and inverse Luma mapping processes, some intermediate variables and data arrays are needed.

[0268] First of all, the variable *OrgCW* is derived as follows:

$$OrgCW = (1 \ll BitDepth) / 16$$

[0269] Then, the variable *lmcsDeltaCW[i]*, with $i = lmcs_min_bin_idx \dots LmcsMaxBinIdx$, is computed as follows:

$$lmcsDeltaCW[i] = (1 - 2 * lmcs_delta_sign_cw_flag[i]) * lmcs_delta_abs_cw[i]$$

[0270] The new variable *lmcsCW[i]* is derived as follows:

[0271] For $i = 0 \dots lmcs_min_bin_idx - 1$, *lmcsCW[i]* is set equal 0.

[0272] For $i = lmcs_min_bin_idx \dots LmcsMaxBinIdx$, the following applies:

$$InputPivot[i] = i * OrgCW$$

[0273] The value of *lmcsCW[i]* shall be in the range of ($OrgCW > 3$) to ($OrgCW < 3 - 1$), inclusive.

[0274] For $i = LmcsMaxBinIdx + 1 \dots 15$, *lmcsCW[i]* is set equal 0.

[0275] The variable *InputPivot[i]*, with $i = 0 \dots 16$, is derived as follows:

$$lmcsCW[i] = OrgCW + lmcsDeltaCW[i]$$

[0276] The variable *LmcsPivot[i]* with $i = 0 \dots 16$, the variables *ScaleCoeff[i]* and *InvScaleCoeff[i]* with $i = 0 \dots 15$, are computed as follows:

$$\begin{aligned} &LmcsPivot[0] = 0; \\ &\text{for}(i = 0; i <= 15; i++) \{ \\ &\quad LmcsPivot[i + 1] = LmcsPivot[i] + lmcsCW[i] \\ &ScaleCoeff[i] = \\ &\quad (lmcsCW[i] * (1 \ll 11) + (1 \ll (\text{Log2}(OrgCW) - 1))) \gg (\text{Log2}(OrgCW)) \\ &\quad \text{if}(lmcsCW[i] == 0) \\ &\quad \quad InvScaleCoeff[i] = 0 \\ &\quad \quad \text{else} \\ &\quad \quad InvScaleCoeff[i] = OrgCW * (1 \ll 11) / lmcsCW[i] \end{aligned}$$

Forward Luma Mapping

[0277] As illustrated by FIG. 8 when the LMCS is applied for Luma, the Luma remapped sample called *predMapSamples[i][j]* is obtained from the prediction sample *predSamples[i][j]*.

[0278] The *predMapSamples[i][j]* is computed as follows:

[0279] First of all, an index *idx Y* is computed from the prediction sample

[0280] *predSamples[i][j]*, at location (i, j)

[0281] $idxY = \text{predSamples}[i][j] \gg \text{Log } 2 (OrgCW)$

[0282] Then *predMapSamples[i][j]* is derived as follows by using the intermediate variables

[0283] *idx Y*, *LmcsPivot[idx Y]* and *InputPivot[idx Y]* of section 0:

$$\begin{aligned} predMapSamples[i][j] &= LmcsPivot[idxY] + (ScaleCoeff[idxY] * \\ &\quad (predSamples[i][j] - InputPivot[idxY]) + (1 \ll 10)) \gg 11 \end{aligned}$$

Luma Reconstruction Samples

[0284] The reconstruction process is obtained from the predicted luma sample *predMapSample[i][j]* and the residual luma samples *resiSamples[i][j]*.

[0285] The reconstructed luma picture sample *recSamples[i][j]* is simply obtained by adding *predMapSample[i][j]* to *resiSamples[i][j]* as follows:

$$recSamples[i][j] = \text{Clip1}(predMapSamples[i][j] + resiSamples[i][j])$$

[0286] In this above relation, the Clip 1 function is a clipping function to make sure that the reconstructed sample is between 0 and $1 \ll BitDepth - 1$.

Inverse Luma Mapping

[0287] When applying the inverse luma mapping according to FIG. 8, the following operations are applied on each sample *recSample[i][j]* of the current block being processed:

[0288] First, an index *idx Y* is computed from the reconstruction sample

[0289] *recSamples[i][j]*, at location (i, j)

[0290] $idxY = \text{recSamples}[i][j] \gg \text{Log } 2 (OrgCW)$

[0291] The inverse mapped luma sample *invLumaSample[i][j]* is derived as follows based on the:

$$\begin{aligned} invLumaSample[i][j] &= InputPivot[idxYInv] + (InvScaleCoeff[idxYInv] * \\ &\quad (recSample[i][j] - LmcsPivot[idxYInv]) + (1 \ll 10)) \gg 11 \end{aligned}$$

[0292] A clipping operation is then done to get the final sample:

[0293] $finalSample[i][j] = \text{Clip1}(invLumaSample[i][j])$

Chroma Scaling

Lmcs Semantics for Chroma Scaling

[0294] The syntax element *lmcs_delta_abs_crs* in Table 6 specifies the absolute codeword value of the variable *lmcs-DeltaCrs*. The value of *lmcs_delta_abs_crs* shall be in the range of 0 and 7, inclusive. When not present, *lmcs_delta_abs_crs* is inferred to be equal to 0.

[0295] The syntax element `lmcs_delta_sign_crs_flag` specifies the sign of the variable `lmcsDeltaCrs`. When not present, `lmcs_delta_sign_crs_flag` is inferred to be equal to 0.

Lmcs Intermediate Variable Computation for Chroma Scaling

[0296] To apply the Chroma scaling process, some intermediate variables are needed.

[0297] The variable `lmcsDeltaCrs` is derived as follows:

$$lmcsDeltaCrs = (1 - 2 * lmcs_delta_sign_crs_flag) * lmcs_delta_abs_crs$$

[0298] The variable `ChromaScaleCoeff[i]`, with $i=0 \dots 15$, is derived as follows:

$$\text{if}(lmcsCW[i] == 0)$$

$$ChromaScaleCoeff[i] = (1 \ll 11)$$

else

$$ChromaScaleCoeff[i] = OrgCW * (1 \ll 11) / (lmcsCW[i] + lmcsDeltaCrs)$$

Chroma Scaling Process

[0299] In a first step, the variable `invAvgLuma` is derived in order to compute the average luma value of reconstructed Luma samples around the current corresponding Chroma block. The average Luma is computed from left and top luma block surrounding the corresponding Chroma block. If no sample is available the variable `invAvgLuma` is set as follows:

$$invAvgLuma = 1 \ll (BitDepth - 1)$$

[0300] Based on the intermediate arrays `LmcsPivot[ ]` of section 0, the variable `idxYInv` is then derived as follows:

For (`idxYInv =`

`lmcs_min_bin_idx; idxYInv <= LmcsMaxBinIdx; idxYInv++`){

`if(invAvgLuma < LmcsPivot[idxYInv + 1]) break`

}

`idxYInv = Min(idxYInv, 15)`

[0301] The variable `varScale` is derived as follows:

[0302] `varScale=ChromaScaleCoeff[idx YInv]`

[0303] When a transform is applied on the current Chroma block, the reconstructed Chroma picture sample array `recSamples` is derived as follows

$$recSamples[i][j] = Clip1(predSamples[i][j] +$$

$$Sign(resiSamples[i][j]) * varScale + (1 \ll 10) \gg 11))$$

[0304] If no transform has been applied for the current block, the following applies:

[0305] `recSamples[i][j]=Clip1(predSamples[i][j])`

Encoder Consideration

[0306] The basic principle of an LMCS encoder is to first assign more codewords to ranges where those dynamic range segments have lower codewords than the average variance. In an alternative formulation of this, the main target of LMCS is to assign fewer codewords to those dynamic range segments that have higher codewords than the average variance. In this way, smooth areas of the picture will be coded with more codewords than average, and vice versa.

[0307] All the parameters (see Table 6) of the LMCS tools which are stored in the APS are determined at the encoder side. The LMCS encoder algorithm is based on the evaluation of local luma variance and is optimizing the determination of the LMCS parameters according to the basic principle described above. The optimization is then conducted to get the best PSNR metrics for the final reconstructed samples of a given block.

Embodiments where Information is Signalled in the Picture Header and not in the Slice Header

[0308] In embodiments, the signalling of an information which can be signalled into the picture header or in the slice header is signalled in the picture header and not in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the signalling of an information, which can be signalled into the picture header or in the slice header, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the signalling of an information, which can be signalled into the picture header or in the slice, is signalled in the picture header. Table 12 illustrates an implementation of embodiments where a tool name is replaced by XXX. In this table, the signalling of parameters is not authorized in the slice header when the picture header is signalled in the slice header.

TABLE 12

Summary of signalling		
<code>xxx_info_in_ph_flag = 1 xxx_info_in_ph_flag = 0</code>		
<code>picture_header_in_slice_header_flag = 1</code>	XXX in Picture Header	Not Applicable
<code>picture_header_in_slice_header_flag = 0</code>	XXX in Picture Header	XXX in Slice Header or not present

[0309] In one embodiment, the following condition is added in the semantics of `xxx_info_in_ph_flag`:

[0310] “When slice headers referring to the PPS contains the PH syntax structure, it is a requirement of bitstream conformance that `XXX_info_in_ph_flag` shall be equal to 1.”

[0311] And/or

[0312] “When `XXX_info_in_ph_flag` is equal to 0, the `picture_header_in_slice_header_flag` shall be equal to 0”

[0313] When the picture header is in the slice header this implies only one slice is present for the current picture. Thus, having the information transmitted or transmittable for both slice and picture doesn’t increase the flexibility for encoder or decoder as the parameters will be the same. In other words, if the information is in the picture header, corresponding information in the slice header will be redundant. Similarly, if the information is in the slice header, corresponding information in the picture header will be redundant. By enforcing a condition so that such information may be in the picture header and not in the slice header in a case where the picture header is in the slice header, the decoder implementation may be simplified by limiting the redundancies in the signaling.

QP (Quantization Parameter) Delta

[0314] In an embodiment, the QP delta signalling is avoided in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the QP delta signalling, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the QP delta is signalled in the picture header. In other words, the tool XXX mentioned above is QP Delta.

[0315] Table 13 illustrates a way that this may be implemented so that signalling of QP delta information is not authorized (i.e. not permitted) when the picture header is signalled in the slice header.

[0316] When the picture header is in the slice header (implying only one slice is present for the current picture) the information being transmittable in slice and picture header doesn’t increase the flexibility for encoder or decoder as the parameters will be the same. So to reduce the decoder implementation complexity it is better to have only one possible signaling QP delta information in one of the slice or picture header to code the same coding possibility, in this case the QP delta parameters (`qp_delta_info`).

[0317] In an implementation of the first embodiment following condition may be added in the semantics of `qp_delta_info_in_ph_flag`:

[0318] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that `qp_delta_info_in_ph_flag` shall be equal to 1.”

[0319] And/or

[0320] “When `qp_delta_info_in_ph_flag` is equal to 0, the `picture_header_in_slice_header_flag` shall be equal to 0”

[0321] In another embodiment, the decoding QP delta parameters in the slice header are authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 0 as depicted in Table 14. Modifications to the syntax are shown in underline. In this table, the slice `qp_delta` information of the slice header can be decoded if QP delta information is signalled at slice level (`qp_delta_info_in_ph_flag` equal to 0) and if the picture header is not transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 0).

TABLE 13

QP delta signalling		
<u><code>qp_delta_info_in_ph_flag</code> = 1 <code>qp_delta_info_in_ph_flag</code> = 0</u>		
<code>picture_header_in_slice_header_flag</code> = 1	QP delta in Picture Header	Not Applicable
<code>picture_header_in_slice_header_flag</code> = 0	QP delta in Picture Header	QP delta in Slice Header or not present

TABLE 14

Partial Slice header showing modification for QP delta	
	Descriptor
slice_header() {	
picture_header_in_slice_header_flag	u(1)
if(picture_header_in_slice_header_flag)	
picture_header_structure()	
...	
if(!qp_delta_info_in_ph_flag && !picture_header_in_slice_header_flag)	
slice_qp_delta	se(v)
...	
}	

[0322] In an embodiment, the decoding QP delta parameters in the Picture header are systematically authorized when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 15. According to this table, the QP delta information of the slice header can be decoded only if QP delta information is signalled in the picture header (`qp_delta_info_in_ph_flag` equal to 1) or if the picture header is transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 1).

TABLE 15

Partial Picture header showing modification for QP Delta	
	Descriptor
picture_header_structure() {	
...	
if(qp_delta_info_in_ph_flag picture_header_in_slice_header_flag)	
ph_qp_delta	se(v)
...	
}	

Reference Picture List (RPL)

[0323] In one embodiment, the signalling of reference picture lists is avoided in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the signalling of reference picture lists, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the signalling of reference picture lists is signalled in the picture header. In other words, the tool XXX mentioned above is RPL. In other words, the tool XXX mentioned above is RPL.

[0324] Table 16 illustrates shows an implementation of this embodiment where RPL signalling is not authorized when the picture header is signalled in the slice header.

TABLE 16

Summary of RPL signalling.		
	<code>rpl_info_in_ph_flag = 1</code>	<code>rpl_info_in_ph_flag = 0</code>
<code>picture_header_in_slice_header_flag = 1</code>	RPL in Picture Header	Not Applicable
<code>picture_header_in_slice_header_flag = 0</code>	RPL in Picture Header	RPL in Slice Header or not present

[0325] When the picture header is in the slice header (implying only one slice is present for the current picture) the information being transmittable in slice and picture header doesn't increase the flexibility for encoder or decoder as the parameters will be the same. According to this embodiment, the decoder implementation complexity is reduced because it is better to have only one possible signaling of information, in one of the slice or picture header to code the same coding possibility, in this case the reference picture list (RPL) information (`rpl_info`).

[0326] In an embodiment, the following condition is added in the semantics of `rpl_info_in_ph_flag`:

[0327] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that `rpl_info_in_ph_flag` shall be equal to 1.”

[0328] And/or

[0329] “When `rpl_info_in_ph_flag` is equal to 0, the `picture_header_in_slice_header_flag` shall be equal to 0”

[0330] In an embodiment, the decoding of reference picture lists parameters in the slice header is authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 0 as depicted in Table 17. In this table, the `ref_pic_lists()` information of the slice header can be decoded if the reference picture lists information is signalled at slice level (`rpl_info_in_ph_flag` equal to 0) and if the picture header is not transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 0).

[0331] In an embodiment, the temporal parameters `slice_collocated_from_l0_flag` and `slice_collocated_ref_idx` can't be decoded when the picture header is in the slice header as depicted in Table 17.

TABLE 17

Partial Slice header showing modification for RPL	Descriptor
<pre> slice_header() { picture_header_in_slice_header_flag if(picture_header_in_slice_header_flag) picture_header_structure() ... if(!rpl_info_in_ph_flag && !picture_header_in_slice_header_flag && ((nal_unit_type != IDR_W_RADL && nal_unit_type != IDR_N_LP) sps_idr_rpl_present_flag)) ref_pic_lists() if((rpl_info_in_ph_flag ((nal_unit_type != IDR_W_RADL && nal_unit_type != IDR_N_LP) sps_idr_rpl_present_flag)) && (slice_type != I && num_ref_entries[0][RplIdx[0]] > 1) (slice_type == B && num_ref_entries[1][RplIdx[1]] > 1)) { num_ref_idx_active_override_flag if(num_ref_idx_active_override_flag) for(i = 0; i < (slice_type == B ? 2 : 1); i++) if(num_ref_entries[i][RplIdx[i]] > 1) num_ref_idx_active_minus1[i] } if(slice_type != I) { if(cabac_init_present_flag) cabac_init_flag if(ph_temporal_mvp_enabled_flag && !rpl_info_in_ph_flag && !picture_header_in_slice_header_flag) { if(slice_type == B) slice_collocated_from_j0_flag if((slice_collocated_from_j0_flag && NumRefIdxActive[0] > 1) (! slice_collocated_from_j0_flag && NumRefIdxActive[1] > 1)) slice_collocated_ref_idx } } } } </pre>	u(1) u(1) ue(v) u(1) ue(v)

[0332] In an embodiment, the decoding of RPL parameters in the Picture header is systematically authorized when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 18. In this table the RPL information of the picture header can be decoded only if RPL information is signalled at picture level (`rpl_info_in_ph_flag` equal to 1) or if the picture header is transmitted in

the slice header (picture_header_in_slice_header_flag equal to 1).

[0333] In an embodiment, the temporal parameters `ph_collocated_from_10_flag` and `ph_collocated_ref_idx` can be decoded when the picture header is in the slice header as depicted in Table 18.

TABLE 18

Partial Picture header showing modification for RPL	
	Descriptor
<pre> picture_header_structure() { ... if(rpl_info_in_ph_flag <u>picture_header_in_slice_header_flag</u>) ref_pic_lists() ... if(ph_temporal_mvp_enabled_flag && (rpl_info_in_ph_flag <u>picture_header_in_slice_header_flag</u>)) { ph_collocated_from_i0_flag if((ph_collocated_from_i0_flag && num_ref_entries[0][RplIdx[0]] > 1) (!ph_collocated_from_i0_flag && num_ref_entries[1][RplIdx[1]] > 1)) ph_collocated_ref_idx } ... } </pre>	<pre> u(1) ue(v) </pre>

Deblocking Filter (DBF)

[0334] In an embodiment, the signalling of deblocking filter parameters is avoided in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the signalling of deblocking filter parameters, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the signalling of deblocking filter parameters is signalled in the picture header. In other words, the tool XXX mentioned above is DBF. In other words, the tool XXX mentioned above is DBF.

[0335] Table 19 illustrates an implementation according to this embodiment where DBF signalling is not authorized when the picture header is signalled in the slice header.

TABLE 19

Summary of DBF signalling		
	dbf_info_in_ph_flag = 1	dbf_info_in_ph_flag = 0
picture_header_in_slice_header_flag = 1	DBF in Picture Header	Not Applicable
picture_header_in_slice_header_flag = 0	DBF in Picture Header	DBF in Picture Header

[0336] When the picture header is in the slice header, as only one slice is present for the current picture, the information transmitted in slice or picture doesn't increase the flexibility for encoder or decoder as the parameters are the same. So to reduce the decoder implementation complexity it is better to have only one possible signaling of DBF information to code the same coding possibility.

[0337] In an embodiment, the following condition is added in the semantics of dbf_info_in_ph_flag:

[0338] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that dbf_info_in_ph_flag shall be equal to 1.”

[0339] And/or

[0340] “When dbf_info_in_ph_flag is equal to 0, the picture_header_in_slice_header_flag shall be equal to 0”

[0341] In an embodiment, the DBF parameters in the slice header are authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 0 as depicted in Table 20. In this table, the `slice_deblocking_filter_override_flag` flag of the slice header can be decoded if DBF information is signalled at slice level (`dbf_info_in_ph_flag` equal to 0) and if the picture header is not transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 0).

TABLE 20

Partial Slice header showing modification for DBF	
	Descriptor
slice_header() { picture_header_in_slice_header_flag if(picture_header_in_slice_header_flag) picture_header_structure() ... if(deblocking_filter_override_enabled_flag && !dbf_info_	u(1)

TABLE 20-continued

Partial Slice header showing modification for DBF	
	Descriptor
in_ph_flag && !picture_header_in_slice_header_flag)	
slice_deblocking_filter_override_flag	u(1)
if(slice_deblocking_filter_override_flag) {	
slice_deblocking_filter_disabled_flag	u(1)
if(!slice_deblocking_filter_disabled_flag) {	
... }	
}	
... }	
}	

[0342] In an embodiment, the DBF parameters in the picture header is systematically authorized when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 21. In this table the DBF information of the slice header can be decoded only if DBF information is signalled in the picture header (`dbf_info_in_ph_flag` equal to 1) or if the picture header is transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 1).

TABLE 21

Partial Picture header showing modification for DBF	
Descriptor	
<pre> picture_header_structure() { ... if(deblocking_filter_override_enabled_flag && (dbf_info_ in_ph_flag <u>picture_header_in_slice_header_flag</u>)) { ph_deblocking_filter_override_flag if(ph_deblocking_filter_override_flag) { ph_deblocking_filter_disabled_flag if(!ph_deblocking_filter_disabled_flag) { ... } } } } </pre>	<pre> u(1) u(1) </pre>

SAO (Sample Adaptive Offset)

[0343] In one embodiment, the SAO signalling is avoided in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the SAO signalling, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the SAO signalling is signalled in the picture header. In other words, the tool XXX mentioned above is SAO.

[0344] Table 22 illustrates this embodiment where SAO signalling is not authorized when the picture header is signalled in the slice header.

TABLE 22

Summary of SAO signalling		
	sao_info_in_ph_flag = 1	sao_info_in_ph_flag = 0
picture_header_in_slice_header_flag = 1	SAO in Picture Header	Not Applicable
picture_header_in_slice_header_flag = 0	SAO in Picture Header	SAO in Slice Header or not present

[0345] When the picture header is in the slice header, as only one slice is present for the current picture, the information transmitted in slice or picture doesn't increase the flexibility for encoder or decoder as the parameters are the same. So to reduce the decoder implementation complexity it is better to have only one possible signaling of SAO information to code the same coding possibility.

[0346] In an embodiment, the following condition is added in the semantics of sao_info_in_ph_flag:

[0347] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that sao_info_in_ph_flag shall be equal to 1.” And/or

[0348] “When sao_info_in_ph_flag is equal to 0, the picture_header_in_slice_header_flag shall be equal to 0”

[0349] In an embodiment, the decoding SAO parameters in the slice header is authorized only when the value of the flag picture_header_in_slice_header_flag is set equal to 0 as depicted in

[0350] Table 23. In this table the slice_sao_luma_flag flag of the slice header can be decoded if SAO information is signalled at slice level (sao_info_in_ph_flag equal to 0) and if the picture header is not transmitted in the slice header (picture_header_in_slice_header_flag equal to 0).

TABLE 23

Partial Slice header showing modification for SAO	
	Descriptor
slice_header() {	
picture_header_in_slice_header_flag	u(1)
if(picture_header_in_slice_header_flag)	
picture_header_structure()	
...	
if(sps_sao_enabled_flag && !sao_info_in_ph_flag && !picture_header_in_slice_header_flag) {	
slice_sao_luma_flag	u(1)
if(ChromaArrayType != 0)	
slice_sao_chroma_flag	u(1)
}	
...	
}	

[0351] In an embodiment, the decoding SAO parameters in the Picture header is systematically authorized when the value of the flag picture_header_in_slice_header_flag is set equal to 1 as depicted in Table 24. In this table the SAO information of the slice header can be decoded only if SAO information is signalled in the picture header (sao_info_in_ph_flag equal to 1) or if the picture header is transmitted in the slice header (picture_header_in_slice_header_flag equal to 1).

TABLE 24

Partial Picture header showing modification for SAO	
	Descriptor
picture_header_structure() {	
...	
if(sps_sao_enabled_flag && (sao_info_in_ph_flag picture_header_in_slice_header_flag)) {	
ph_sao_luma_enabled_flag	u(1)
if(ChromaArrayType != 0)	
ph_sao_chroma_enabled_flag	u(1)
}	
...	
}	

Weighted Prediction (WP)

[0352] In one embodiment, the signalling of the weighted prediction is avoided in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the signalling of the weighted prediction, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the signalling of the weighted prediction is signalled in the picture header. In other words, the tool XXX mentioned above is WP.

[0353] As shown in Table 25, WP parameters can be signalled in the slice header when wp_info_in_ph_flag is set equal to 0 with the current syntax.

[0354] Table 25 illustrates an implementation of this embodiment where WP signalling is not authorized when the picture header is signalled in the slice header.

TABLE 25

Summary of WP signalling		
wp_info_in_ph_flag = 1 wp_info_in_ph_flag = 0		
picture_header_in_slice_header_flag = 1	WP in Picture Header	Not Applicable
picture_header_in_slice_header_flag = 0	WP in Picture Header	WP in Slice Header or not present

[0355] When the picture header is in the slice header, as only one slice is present for the current picture, the information transmitted in slice or picture doesn't increase the flexibility for encoder or decoder as the parameters are the same. So to reduce the decoder implementation complexity it is better to have only one possible signaling of WP info to code the same coding possibility.

[0356] In an embodiment, the following condition is added in the semantics of wp_info_in_ph_flag:

[0357] "When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that wp_info_in_ph_flag shall be equal to 1."

[0358] And/or

[0359] "When wp_info_in_ph_flag is equal to 0, the picture_header_in_slice_header_flag shall be equal to 0"

[0360] In an embodiment, the decoding WP parameters in the slice header is authorized only when the value of the flag picture_header_in_slice_header_flag is set equal to 0 as depicted in Table 26. In this table the pred_weight_table() function containing weighted prediction parameters can be decoded if WP information is signalled at slice level (wp_info_in_ph_flag equal to 0) and if the picture header is not transmitted in the slice header (picture_header_in_slice_header_flag equal to 0).

TABLE 26

Partial Slice header showing modification for WP	
	Descriptor
slice_header() { picture_header_in_slice_header_flag if(picture_header_in_slice_header_flag) picture_header_structure() ... if(!wp_info_in_ph_flag && !picture_header_in_slice_header_flag && ((pps_weighted_pred_flag && slice_type == P) (pps_weighted_bipred_flag && slice_type == B))) pred_weight_table() ... }	u(1)

[0361] In an embodiment, the decoding WP parameters in the Picture header are systematically authorized when the value of the flag picture_header_in_slice_header_flag is set equal to 1 as depicted in Table 27. In this table the WP information can be decoded only if WP information is signalled in the picture header (wp_info_in_ph_flag equal to 1) or if the picture header is transmitted in the slice header (picture_header_in_slice_header_flag equal to 1).

TABLE 27

Partial Picture header showing modification for WP	
	Descriptor
picture_header_structure() { ... if((pps_weighted_pred_flag pps_weighted_bipred_flag) && (wp_info_in_ph_flag picture_header_in_slice_header_flag)) pred_weight_table() ... }	u(1)

ALF

[0362] In one embodiment, the signalling of ALF is avoided in the slice header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the signalling of ALF, is signalled in the slice header. In another equivalent way, the picture header is signalled in the slice header when the signalling of ALF is signalled in the picture header. In other words, the tool XXX mentioned above is ALF.

[0363] Indeed, currently when the picture header is signalled in the slice header (picture_header_in_slice_header_flag=1) and when the ALF is signalled in the slice header (alf_info_in_ph_flag=0), all the parameters of the picture header should be parsed before obtaining the ALF APS ID. Consequently, it increases the complexity of the parsing for some streaming applications as all variables of PPS, SPS, picture header etc., should be keep in memory to parse the ALF APS ID.

[0364] Moreover, when the picture header is in the slice header, as only one slice is present for the current picture, the information transmitted in slice or picture doesn't increase the flexibility for encoder or decoder as the parameters are the same. So to reduce the decoder implementation complexity it is better to have only one possible signaling to code the same coding possibility. Table 28 illustrates this embodiment where ALF signalling is not authorized when the picture header is signalled in the slice header.

TABLE 28

Summary of ALF signalling		
alf_info_in_ph_flag = 1 alf_info_in_ph_flag = 0		
picture_header_in_slice_header_flag = 1	ALF in Picture Header	Not Applicable
picture_header_in_slice_header_flag = 0	ALF in Picture Header	ALF in Slice Header or not present

[0365] In an embodiment, the following condition is added in the semantics of alf_info_in_ph_flag:

[0366] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that alf_info_in_ph_flag shall be equal to 1.”

[0367] And/or

[0368] “When alf_info_in_ph_flag is equal to 0, the picture_header_in_slice_header_flag shall be equal to 0”

[0369] In an embodiment, the decoding ALF parameters in the slice header is authorized only when the value of the flag picture_header_in_slice_header_flag is set equal to 0 as depicted in Table 29. In this table the ALF information of the slice header can be decoded only if ALF is enabled at SPS level (sps_alf_enabled_flag equal to 1) and if ALF information is signalled at slice level (alf_info_in_ph_flag equal to

0) and if the picture header is not transmitted in the slice header (picture_header_in_slice_header_flag equal to 0).

TABLE 29

Partial Slice header showing modification for ALF	
	Descriptor
slice_header() {	
picture_header_in_slice_header_flag	u(1)
if(picture_header_in_slice_header_flag)	
picture_header_structure()	
...	
if(sps_alf_enabled_flag && !alf_info_in_ph_flag	
&& !picture_header_in_slice_header_flag) {	
slice_alf_enabled_flag	u(1)
if(slice_alf_enabled_flag) {	
slice_num_alf_aps_ids_luma	u(3)
for(i = 0; i < slice_num_alf_aps_ids_luma; i++)	
slice_alf_aps_id_luma[i]	u(3)
if(ChromaArrayType != 0)	
slice_alf_chroma_idc	u(2)
if(slice_alf_chroma_idc)	
slice_alf_aps_id_chroma	u(3)
if(sps_ccalf_enabled_flag) {	
slice_cc_alf_cb_enabled_flag	u(1)
if(slice_cc_alf_cb_enabled_flag)	
slice_cc_alf_cb_aps_id	u(3)

TABLE 29-continued

Partial Slice header showing modification for ALF	
	Descriptor
slice_cc_alf_cr_enabled_flag	u(1)
if(slice_cc_alf_cr_enabled_flag)	
slice_cc_alf_cr_aps_id	u(3)
}	
}	
}	
...	
}	

[0370] In an embodiment, the decoding ALF parameters in the Picture header is systematically authorized when the value of the flag picture_header_in_slice_header_flag is set equal to 1 as depicted in Table 30. In this table the ALF information of the slice header can be decoded only if ALF is enabled at SPS level (sps_alf_enabled_flag equal to 1) and

if ALF information is signalled at picture level (alf_info_in_ph_flag equal to 1) or if the picture header is transmitted in the slice header (picture_header_in_slice_header_flag equal to 1).

TABLE 30

Partial Picture header showing modification for ALF	
	Descriptor
picture_header_structure() { ... if(sps_alf_enabled_flag && (alf_info_in_ph_flag picture_header_in_slice_header_flag)) { ph_alf_enabled_flag if(ph_alf_enabled_flag) { ph_num_alf_aps_ids_luma for(i = 0; i < ph_num_alf_aps_ids_luma; i++) ph_alf_aps_id_luma[i] if(ChromaArrayType != 0) ph_alf_chroma_idc if(ph_alf_chroma_idc > 0) ph_alf_aps_id_chroma if(sps_cc_alf_enabled_flag) { ph_cc_alf_cb_enabled_flag if(ph_cc_alf_cb_enabled_flag) ph_cc_alf_cb_aps_id ph_cc_alf_cr_enabled_flag if(ph_cc_alf_cr_enabled_flag) ph_cc_alf_cr_aps_id } } } }	 u(1) u(3) u(3) u(2) u(3) u(1) u(3) u(1) u(3)

All Tools/Parameters

[0371] In one embodiment, when the picture header is signalled in the slice header, all tools (and/or parameters) which can be signalled in the picture header or in the slice header are restricted to being signalled in the picture header. As mention in the above description, in an embodiment, the related tools are: QP delta information, reference picture lists, deblocking filter, SAO Weighted prediction and ALF. Other tools may be possible, however, where they are able to be signalled in both the slice and picture header.

[0372] This can be expressed by adding the following constraint:

[0373] “When at least one of the flags rpl_info_in_ph_flag, dbf_info_in_ph_flag, sao_info_in_ph_flag, alf_info_in_ph_flag, wp_info_in_ph_flag, qp_delta_info_in_ph_flag is set equal to 0, the value of picture_header_in_slice_header_flag shall be equal to 0.”

[0374] And/Or by adding:

[0375] “When picture_header_in_slice_header_flag is equal to 1, the flags rpl_info_in_ph_flag, dbf_info_in_ph_flag, sao_info_in_ph_flag, wp_info_in_ph_flag, qp_delta_info_in_ph_flag shall be equal to 1.”

[0376] And/Or by adding to each XXX_info_in_ph_flag the following constraint:

[0377] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that XXX_info_in_ph_flag shall be equal to 1.”

[0378] When all these parameters are signalled in the same header it reduces the decoder implementation complexity as it is better to have only one possible signaling to code the same coding possibility.

Signalling Order Embodiment

[0379] In one alternatives embodiment of the previous ones related to ALF, the information related to ALF APS ID in the slice header is set before the picture header structure as depicted in Table 31. With this embodiment, when ALF is signalled in the slice header and when the picture header is signalled in the slice header, the APS ID can be obtained quickly without parsed all the parameters of the picture header.

TABLE 31

Partial Slice header showing modifications	
	Descriptor
slice_header() { if(sps_alf_enabled_flag && !alf_info_in_ph_flag) { slice_alf_enabled_flag if(slice_alf_enabled_flag) { slice_num_alf_aps_ids_luma for(i = 0; i < slice_num_alf_aps_ids_luma; i++) slice_alf_aps_id_luma[i] if(ChromaArrayType != 0) slice_alf_chroma_idc if(slice_alf_chroma_idc) slice_alf_aps_id_chroma if(sps_ccalf_enabled_flag) { slice_cc_alf_cb_enabled_flag if(slice_cc_alf_cb_enabled_flag) slice_cc_alf_cb_aps_id slice_cc_alf_cr_enabled_flag if(slice_cc_alf_cr_enabled_flag) slice_cc_alf_cr_aps_id } } }	 u(1) u(3) u(3) u(2) u(3) u(1) u(3) u(1) u(3)

TABLE 31-continued

Partial Slice header showing modifications	
	Descriptor
picture_header_in_slice_header_flag	u(1)
if(picture_header_in_slice_header_flag)	
picture_header_structure()	
....	
}	

Number of Weights Embodiment

[0380] In an embodiment when the picture header is in the slice header, the number of weights for the weighted prediction for each list L0 L1 is decoded as depicted in the partial table of syntax element Table 32. Accordingly, the signalling of the number of weights may be restricted to the picture header.

TABLE 32

Partial Weighted prediction parameters syntax table	
	Descriptor
pred_weight_table() {	
...	
if(wp_info_in_ph_flag picture_header_in_slice_header_flag)	
num_l0_weights	ue(v)
...	
if(pps_weighted_bipred_flag && (wp_info_in_ph_flag picture_header_in_slice_header_flag))	
num_l1_weights	ue(v)
...	
}	

Summary of Embodiments where Information is Signalled in the Slice Header and not in the Picture Header

[0381] In one embodiment, the signalling of an information which can be signalled into the picture header or in the slice header is signalled in the slice header and not in the picture header when the picture header is signalled in the slice header. In an equivalent way, the picture header is not signalled in the slice header when the signalling of an information, which can be signalled into the picture header or in the slice header, is signalled in the picture header. In another equivalent way, the picture header is signalled in the slice header when the signalling of an information, which can be signalled into the picture header or in the slice header, is signalled in the slice header. Table 33 illustrates this embodiment where a tool (or parameter) name is replaced by XXX. In this table, the signalling of parameters is not authorized in the picture header when the picture header is signalled in the slice header.

TABLE 33

Summary of a tool signalling.		
	xxx_info_in_ph_flag = 1	xxx_info_in_ph_flag = 0
picture_header_in_slice_header_flag = 1	Not Applicable	XXX in Slice Header or not present
picture_header_in_slice_header_flag = 0	XXX in Picture Header	XXX in Slice Header or not present

[0382] In one embodiment, the following condition is added in the semantics of xxx_info_in_ph_flag:

[0383] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that XXX_info_in_ph_flag shall be equal to 0.”

[0384] And/or

[0385] “When XXX_info_in_ph_flag is equal to 1, the picture_header_in_slice_header_flag shall be equal to 0”

[0386] When the picture header is in the slice header this implies only one slice is present for the current picture. Thus, having the information transmitted or transmittable for both slice and picture doesn’t increase the flexibility for encoder or decoder as the parameters will be the same. In other words, if the information is in the picture header, corresponding information in the slice header will be redundant. Similarly, if the information is in the slice header, corresponding information in the picture header will be redundant. Embodiments described herein simplify the decoder implementation by limiting the redundancies in the signaling of the same coding. In particular, in embodiments, the information is permitted to be in the slice header but not the picture header when the picture header is signalled in the slice header.

QP Delta

[0387] In one embodiment the tool XXX is the QP delta. In an embodiment, the decoding QP delta parameters in the slice header is authorized when the value of the flag picture_header_in_slice_header_flag is set equal to 1 as depicted in Table 33. In an equivalent way, picture_header_in_slice_header_flag is set equal to 0, when the QP delta parameters are signalled in the picture header as depicted in Table 33. In another equivalent way, the picture_header_in_slice_header_flag is set equal to 1, when the QP delta parameters are signalled in the slice header as depicted in Table 33. In this table the slice_qp_delta information of the slice header can be decoded if QP delta information is signalled at slice level (qp_delta_info_in_ph_flag equal to 0) or if the picture header is transmitted in the slice header (picture_header_in_slice_header_flag equal to 1).

[0388] This can be obtained by adding to the semantics:

[0389] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that qp_delta_info_in_ph_flag shall be equal to 0.” And/or

[0390] “When qp_delta_info_in_ph_flag is equal to 1, the picture_header_in_slice_header_flag shall be equal to 0”

[0391] In an embodiment, the decoding QP delta parameters in the picture header is systematically avoided when

the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 35. In this table the QP delta information of the picture header can be decoded only if QP delta information is signalled in the picture header (`qp_delta_info_in_ph_flag` equal to 1) and if `picture_header_in_slice_header_flag` is set equal to 0.

Rpl (Reference Picture List)

[0392] In one embodiment the tool XXX is the reference picture lists. In an embodiment, the decoding of reference picture lists parameters in the slice header is authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 33. In an equivalent way, `picture_header_in_slice_header_flag` is set equal to 0, when the reference picture lists parameters are signalled in the picture header as depicted in Table 33. In another equivalent way, the `picture_header_in_slice_header_flag` is set equal to 1, when the reference picture lists parameters are signalled in the slice header as depicted in Table 33. In this table, the `ref_pic_lists()` information of the slice header can be decoded if the reference picture lists information is signalled at slice level (`rpl_info_in_ph_flag` equal to 0) and if the picture header is transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 1).

[0393] In an embodiment, the temporal parameters `slice_collocated_from_l0_flag` and `slice_collocated_ref_idx` can be decoded when the picture header is in the slice header as depicted in Table 34.

[0394] In an embodiment, the decoding of RPL parameters in the Picture header is systematically avoided when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 35. In this table the RPL information of the picture header can be decoded only if RPL information is signalled at picture level (`rpl_info_in_ph_flag` equal to 1) and if `picture_header_in_slice_header_flag` is set equal to 0.

[0395] In an embodiment, the temporal parameters `ph_collocated_from_l0_flag` and `ph_collocated_ref_idx` can't be decoded when the picture header is in the slice header as depicted in Table 35.

Deblocking Filter (DBF)

[0396] In one embodiment the tool XXX is the deblocking filter (DBF). In one alternative or additional embodiment, the DBF parameters in the slice header are authorized when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 33. In an equivalent way, `picture_header_in_slice_header_flag` is set equal to 0, when the DBF parameters are signalled in the picture header as depicted in Table 33. In another equivalent way, the `picture_header_in_slice_header_flag` is set equal to 1, when the DBF parameters are signalled in the slice header as depicted in Table 33. In this table the `slice_deblocking_filter_override_flag` flag of the slice header can be decoded if DBF information is signalled at slice level (`dbf_info_in_ph_flag` equal to 0) or if the picture header is transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 1).

[0397] In an embodiment, the DBF parameters in the picture header are systematically avoided when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 35. In this table the DBF information of the picture header can be decoded only if DBF informa-

tion is signalled in the picture header (`dbf_info_in_ph_flag` equal to 1) and if `picture_header_in_slice_header_flag` is equal to 0.

Sample Adaptive Offset (SAO)

[0398] In one embodiment the tool XXX is SAO (Sample Adaptive Offset). In one alternative or additional embodiment, the SAO parameters in the slice header are authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 33. In an equivalent way, `picture_header_in_slice_header_flag` is set equal to 0, when the SAO parameters are signalled in the picture header as depicted in Table 33. In another equivalent way, the `picture_header_in_slice_header_flag` is set equal to 1, when the SAO parameters are signalled in the slice header as depicted in Table 33. In this table the `slice_sao_luma_flag` flag of the slice header can be decoded if SAO information is signalled at slice level (`sao_info_in_ph_flag` equal to 0) and if the picture header is transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 1).

[0399] In an embodiment, the SAO parameters in the picture header are systematically avoided when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 35. In this table the SAO information of the picture header can be decoded only if SAO information is signalled in the picture header (`sao_info_in_ph_flag` equal to 1) and if `picture_header_in_slice_header_flag` is set equal to 0.

WP (Weighted Prediction)

[0400] In one embodiment the tool XXX is the WP (Weighted prediction). In an embodiment, the WP parameters in the slice header are authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 33. In an equivalent way, `picture_header_in_slice_header_flag` is set equal to 0, when the WP parameters are signalled in the picture header as depicted in Table 33. In another equivalent way, the `picture_header_in_slice_header_flag` is set equal to 1, when the WP parameters are signalled in the slice header as depicted in Table 33. In this table the `pred_weight_table()` function containing weighted prediction parameters can be decoded if WP information is signalled at slice level (`wp_info_in_ph_flag` equal to 0) or if the picture header is transmitted in the slice header (`picture_header_in_slice_header_flag` equal to 1).

[0401] In one additional embodiment, the decoding WP parameters in the Picture header is systematically avoided when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 35. In this table the WP information can be decoded only if WP information is signalled in the picture header (`wp_info_in_ph_flag` equal to 1) and if `picture_header_in_slice_header_flag` is set equal to 0.

ALF (Adaptive Loop Filter)

[0402] In one embodiment the tool XXX is the ALF (Adaptive loop filter). In an embodiment, the decoding ALF parameters in the slice header is authorized only when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 33. . . . In an equivalent way, `picture_header_in_slice_header_flag` is set equal to 0, when the ALF parameters are signalled in the picture header as depicted in Table 33. In another equivalent way, the `picture_`

header_in_slice_header_flag is set equal to 1, when the ALF parameters are signalled in the slice header as depicted in Table 33 In this table the ALF information of the slice header can be decoded only if ALF is enabled at SPS level (sps_alf_enabled_flag equal to 1) and if ALF information is signalled at slice level (alf_info_in_ph_flag equal to 0) or if the picture header is transmitted in the slice header (picture_header_in_slice_header_flag equal to 1).

[0403] In an embodiment, the decoding ALF parameters in the picture header is systematically avoided when the value of the flag `picture_header_in_slice_header_flag` is set equal to 1 as depicted in Table 35. In this table the ALF information of the slice header can be decoded only if ALF is enabled at SPS level (`sps_alf_enabled_flag` equal to 1) and if ALF information is signalled at picture level (`alf_info_in_ph_flag` equal to 1) and if `picture_header_in_slice_header_flag` is set equal to 0.

TABLE 34

Partial Slice header showing modifications		Descriptor
slice_header() {		
picture_header_in_slice_header_flag		u(1)
if(picture_header_in_slice_header_flag)		
picture_header_structure()		
...		
if(sps_alf_enabled_flag && (!alf_info_in_ph_flag <u>picture_header_in_slice_header_flag</u>))		
}{		
slice_alf_enabled_flag		u(1)
if(slice_alf_enabled_flag) {		
...		
}		
}		
...		
if((!rpl_info_in_ph_flag <u>picture_header_in_slice_header_flag</u>) && ((nal_unit_type		
!=		
IDR_W_RADL && nal_unit_type !=		
IDR_N_LP) sps_idr_rpl_present_flag))		
ref_pic_lists()		
if((rpl_info_in_ph_flag && ! <u>picture_header_in_slice_header_flag</u>) ((nal_unit_type		
!=		
IDR_W_RADL && nal_unit_type !=		
IDR_N_LP) sps_idr_rpl_present_flag)) &&		
(slice_type != I && num_ref_entries[0][RplsIdx[0]] > 1)		
(slice_type == B && num_ref_entries[1][RplsIdx[1]] > 1)) {		
num_ref_idx_active_override_flag		u(1)
if(num_ref_idx_active_override_flag)		
for(i = 0; i < (slice_type == B ? 2: 1); i++)		
if(num_ref_entries[i][RplsIdx[i]] > 1)		
num_ref_idx_active_minus1[i]		ue(v)
}		
...		
if(ph_temporal_mvp_enabled_flag && (!rpl_info_in_ph_flag		
<u>picture_header_in_slice_header_flag</u>)) {		
if(slice_type == B)		
slice_collocated_from_l0_flag		u(1)
if((slice_collocated_from_l0_flag && NumRefIdxActive[0] > 1)		
(! slice_collocated_from_l0_flag && NumRefIdxActive[1] > 1))		
slice_collocated_ref_idx		ue(v)
}		
if((!wp_info_in_ph_flag <u>picture_header_in_slice_header_flag</u>) && ((		
pps_weighted_pred_flag && slice_type == P)		
(pps_weighted_bipred_flag && slice_type == B)))		
pred_weight_table()		
}		
if((!qp_delta_info_in_ph_flag picture_header_in_slice_header_flag))		
slice_qp_delta		se(v)
...		
if(sps_sao_enabled_flag && (!sao_info_in_ph_flag		
<u>picture_header_in_slice_header_flag</u>)) {		
slice_sao_luma_flag		u(1)
if(ChromaArrayType != 0)		
slice_sao_chroma_flag		u(1)
}		
if(deblocking_filter_override_enabled_flag && (!dbf_info_in_ph_flag		
<u>picture_header_in_slice_header_flag</u>))		
slice_deblocking_filter_override_flag		u(1)
...		
}		

TABLE 35

Partial Picture header showing modifications	
	Descriptor
<pre> picture_header_structure() { ... if(sps_alf_enabled_flag && (alf_info_in_ph_flag && !picture_header_in_slice_header_flag)) { ph_alf_enabled_flag if(ph_alf_enabled_flag) { ... } } ... if(rpl_info_in_ph_flag && !picture_header_in_slice_header_flag) ref_pic_lists() ... if(ph_temporal_mvp_enabled_flag && rpl_info_in_ph_flag && !picture_header_in_slice_header_flag) { ph_collocated_from_l0_flag if((ph_collocated_from_l0_flag && num_ref_entries[0][RplIdx[0]] > 1) (!ph_collocated_from_l0_flag && num_ref_entries[1][RplIdx[1]] > 1)) ph_collocated_ref_idx } ... if((pps_weighted_pred_flag pps_weighted_bipred_flag) && wp_info_in_ph_ flag && !picture_header_in_slice_header_flag) pred_weight_table() } if(qp_delta_info_in_ph_flag && !picture_header_in_slice_header_flag) ph_qp_delta ... if(sps_sao_enabled_flag && sao_info_in_ph_flag && !picture_header_in_slice_header_flag) { ph_sao_luma_enabled_flag if(ChromaArrayType != 0) ph_sao_chroma_enabled_flag } ... if(deblocking_filter_override_enabled_flag && dbf_info_in_ph_flag && !picture_header_in_slice_header_flag) { ph_deblocking_filter_override_flag ... } } } </pre>	u(1) u(1) ue(v) se(v) u(1) u(1) u(1)

All Tools/Parameters

[0404] In an embodiment, when the picture header is signalled in the slice header, all tools (and/or parameters) which can be signalled in the picture header or in the slice header are signalled in the slice header. As mention in the above description in an embodiment, the related tools are: QP delta information, reference picture lists, deblocking filter, SAO Weighted prediction and ALF. Other tools may be possible, however, where they are able to be signalled in both the slice and picture header.

[0405] This can be expressed by adding the following constraint:

[0406] “When at least one of the flags `rpl_info_in_ph_flag`, `dbf_info_in_ph_flag`, `sao_info_in_ph_flag`, `alf_info_in_ph_flag`, `wp_info_in_ph_flag`, `qp_delta_info_in_ph_flag` is set equal to 1, the value of `picture_header_in_slice_header_flag` shall be equal to 0.”

[0407] And/Or by adding:

[0408] “When `picture_header_in_slice_header_flag` is equal to 1, the flags `rpl_info_in_ph_flag`, `dbf_info_in_`

`ph_flag`, `sao_info_in_ph_flag`, `wp_info_in_ph_flag`, `qp_delta_info_in_ph_flag` shall be equal to 0.”

[0409] And/Or by adding to each `XXX_info_in_ph_flag` the following constraint:

[0410] “When slice headers referring to the PPS contain the PH syntax structure, it is a requirement of bitstream conformance that `XXX_info_in_ph_flag` shall be equal to 0.”

[0411] When all these parameters are signalling in the same header it reduces the decoder implementation complexity as it is better to have only one possible signaling for tools or parameters where they will inevitably have the same values or properties.

Implementations

[0412] FIG. 10 shows a system **191**, **195** comprising at least one of an encoder **150** or a decoder **100** and a communication network **199** according to embodiments of the present invention. According to an embodiment, the system **195** is for processing and providing a content (for

example, a video and audio content for displaying/outputting or streaming video/audio content) to a user, who has access to the decoder **100**, for example through a user interface of a user terminal comprising the decoder **100** or a user terminal that is communicable with the decoder **100**. Such a user terminal may be a computer, a mobile phone, a tablet or any other type of a device capable of providing/displaying the (provided/streamed) content to the user. The system **195** obtains/receives a bitstream **101** (in the form of a continuous stream or a signal—e.g. while earlier video/audio are being displayed/output) via the communication network **199**.

[0413] According to an embodiment, the system **191** is for processing a content and storing the processed content, for example a video and audio content processed for displaying/outputting/streaming at a later time. The system **191** obtains/receives a content comprising an original sequence of images **151**, which is received and processed (including filtering with a deblocking filter according to the present invention) by the encoder **150**, and the encoder **150** generates a bitstream **101** that is to be communicated to the decoder **100** via a communication network **191**. The bitstream **101** is then communicated to the decoder **100** in a number of ways, for example it may be generated in advance by the encoder **150** and stored as data in a storage apparatus in the communication network **199** (e.g. on a server or a cloud storage) until a user requests the content (i.e. the bitstream data) from the storage apparatus, at which point the data is communicated/streamed to the decoder **100** from the storage apparatus. The system **191** may also comprise a content providing apparatus for providing/streaming, to the user (e.g. by communicating data for a user interface to be displayed on a user terminal), content information for the content stored in the storage apparatus (e.g. the title of the content and other meta/storage location data for identifying, selecting and requesting the content), and for receiving and processing a user request for a content so that the requested content can be delivered/streamed from the storage apparatus to the user terminal. Alternatively, the encoder **150** generates the bitstream **101** and communicates/streams it directly to the decoder **100** as and when the user requests the content. The decoder **100** then receives the bitstream **101** (or a signal) and performs filtering with a deblocking filter according to the invention to obtain/generate a video signal **109** and/or audio signal, which is then used by a user terminal to provide the requested content to the user.

[0414] Any step of the method/process according to the invention or functions described herein may be implemented in hardware, software, firmware, or any combination thereof. If implemented in software, the steps/functions may be stored on or transmitted over, as one or more instructions or code or program, or a computer-readable medium, and executed by one or more hardware-based processing unit such as a programmable computing machine, which may be a PC (“Personal Computer”), a DSP (“Digital Signal Processor”), a circuit, a circuitry, a processor and a memory, a general purpose microprocessor or a central processing unit, a microcontroller, an ASIC (“Application-Specific Integrated Circuit”), a field programmable logic arrays (FPGAs), or other equivalent integrated or discrete logic circuitry. Accordingly, the term “processor” as used herein may refer to any of the foregoing structure or any other structure suitable for implementation of the techniques describe herein.

[0415] Embodiments of the present invention can also be realized by wide variety of devices or apparatuses, including a wireless handset, an integrated circuit (IC) or a set of ICs (e.g. a chip set). Various components, modules, or units are described herein to illustrate functional aspects of devices/apparatuses configured to perform those embodiments, but do not necessarily require realization by different hardware units. Rather, various modules/units may be combined in a codec hardware unit or provided by a collection of interoperative hardware units, including one or more processors in conjunction with suitable software/firmware.

[0416] Embodiments of the present invention can be realized by a computer of a system or apparatus that reads out and executes computer executable instructions (e.g., one or more programs) recorded on a storage medium to perform the modules/units/functions of one or more of the above-described embodiments and/or that includes one or more processing unit or circuits for performing the functions of one or more of the above-described embodiments, and by a method performed by the computer of the system or apparatus by, for example, reading out and executing the computer executable instructions from the storage medium to perform the functions of one or more of the above-described embodiments and/or controlling the one or more processing unit or circuits to perform the functions of one or more of the above-described embodiments. The computer may include a network of separate computers or separate processing units to read out and execute the computer executable instructions. The computer executable instructions may be provided to the computer, for example, from a computer-readable medium such as a communication medium via a network or a tangible storage medium. The communication medium may be a signal/bitstream/carrier wave. The tangible storage medium is a “non-transitory computer-readable storage medium” which may include, for example, one or more of a hard disk, a random-access memory (RAM), a read only memory (ROM), a storage of distributed computing systems, an optical disk (such as a compact disc (CD), digital versatile disc (DVD), or Blu-ray Disc (BD)TM), a flash memory device, a memory card, and the like. At least some of the steps/functions may also be implemented in hardware by a machine or a dedicated component, such as an FPGA (“Field-Programmable Gate Array”) or an ASIC (“Application-Specific Integrated Circuit”).

[0417] FIG. 11 is a schematic block diagram of a computing device **2000** for implementation of one or more embodiments of the invention. The computing device **2000** may be a device such as a micro-computer, a workstation or a light portable device. The computing device **2000** comprises a communication bus connected to: —a central processing unit (CPU) **2001**, such as a microprocessor; —a random access memory (RAM) **2002** for storing the executable code of the method of embodiments of the invention as well as the registers adapted to record variables and parameters necessary for implementing the method for encoding or decoding at least part of an image according to embodiments of the invention, the memory capacity thereof can be expanded by an optional RAM connected to an expansion port for example; —a read only memory (ROM) **2003** for storing computer programs for implementing embodiments of the invention; —a network interface (NET) **2004** is typically connected to a communication network over which digital data to be processed are transmitted or received. The network interface (NET) **2004** can be a single network

interface, or composed of a set of different network interfaces (for instance wired and wireless interfaces, or different kinds of wired or wireless interfaces). Data packets are written to the network interface for transmission or are read from the network interface for reception under the control of the software application running in the CPU **2001**; —a user interface (UI) **2005** may be used for receiving inputs from a user or to display information to a user; —a hard disk (HD) **2006** may be provided as a mass storage device; —an Input/Output module (IO) **2007** may be used for receiving/sending data from/to external devices such as a video source or display. The executable code may be stored either in the ROM **2003**, on the HD **2006** or on a removable digital medium such as, for example a disk. According to a variant, the executable code of the programs can be received by means of a communication network, via the NET **2004**, in order to be stored in one of the storage means of the communication device **2000**, such as the HD **2006**, before being executed. The CPU **2001** is adapted to control and direct the execution of the instructions or portions of software code of the program or programs according to embodiments of the invention, which instructions are stored in one of the aforementioned storage means. After powering on, the CPU **2001** is capable of executing instructions from main RAM memory **2002** relating to a software application after those instructions have been loaded from the program ROM **2003** or the HD **2006**, for example. Such a software application, when executed by the CPU **2001**, causes the steps of the method according to the invention to be performed.

[0418] It is also understood that according to another embodiment of the present invention, a decoder according to an aforementioned embodiment is provided in a user terminal such as a computer, a mobile phone (a cellular phone), a table or any other type of a device (e.g. a display apparatus) capable of providing/displaying a content to a user. According to yet another embodiment, an encoder according to an aforementioned embodiment is provided in an image capturing apparatus which also comprises a camera, a video camera or a network camera (e.g. a closed-circuit television or video surveillance camera) which captures and provides the content for the encoder to encode. Two such examples are provided below with reference to FIGS. **11** and **12**.

Network Camera

[0419] FIG. **11** is a diagram illustrating a network camera system **2100** including a network camera **2102** and a client apparatus **2104**.

[0420] The network camera **2102** includes an imaging unit **2106**, an encoding unit **2108**, a communication unit **2110**, and a control unit **2112**.

[0421] The network camera **2102** and the client apparatus **2104** are mutually connected to be able to communicate with each other via the network **200**.

[0422] The imaging unit **2106** includes a lens and an image sensor (e.g., a charge coupled device (CCD) or a complementary metal oxide semiconductor (CMOS)), and captures an image of an object and generates image data based on the image. This image can be a still image or a video image.

[0423] The encoding unit **2108** encodes the image data by using said encoding methods described above

[0424] The communication unit **2110** of the network camera **2102** transmits the encoded image data encoded by the encoding unit **2108** to the client apparatus **2104**.

[0425] Further, the communication unit **2110** receives commands from client apparatus **2104**. The commands include commands to set parameters for the encoding of the encoding unit **2108**.

[0426] The control unit **2112** controls other units in the network camera **2102** in accordance with the commands received by the communication unit **2110**.

[0427] The client apparatus **2104** includes a communication unit **2114**, a decoding unit **2116**, and a control unit **2118**.

[0428] The communication unit **2114** of the client apparatus **2104** transmits the commands to the network camera **2102**.

[0429] Further, the communication unit **2114** of the client apparatus **2104** receives the encoded image data from the network camera **2102**.

[0430] The decoding unit **2116** decodes the encoded image data by using said decoding methods described above.

[0431] The control unit **2118** of the client apparatus **2104** controls other units in the client apparatus **2104** in accordance with the user operation or commands received by the communication unit **2114**.

[0432] The control unit **2118** of the client apparatus **2104** controls a display apparatus **2120** so as to display an image decoded by the decoding unit **2116**.

[0433] The control unit **2118** of the client apparatus **2104** also controls a display apparatus **2120** so as to display GUI (Graphical User Interface) to designate values of the parameters for the network camera **2102** includes the parameters for the encoding of the encoding unit **2108**.

[0434] The control unit **2118** of the client apparatus **2104** also controls other units in the client apparatus **2104** in accordance with user operation input to the GUI displayed by the display apparatus **2120**.

[0435] The control unit **2119** of the client apparatus **2104** controls the communication unit **2114** of the client apparatus **2104** so as to transmit the commands to the network camera **2102** which designate values of the parameters for the network camera **2102**, in accordance with the user operation input to the GUI displayed by the display apparatus **2120**.

Smart Phone

[0436] FIG. **12** is a diagram illustrating a smart phone **2200**.

[0437] The smart phone **2200** includes a communication unit **2202**, a decoding unit **2204**, a control unit **2206**, display unit **2208**, an image recording device **2210** and sensors **2212**.

[0438] The communication unit **2202** receives the encoded image data via network **200**.

[0439] The decoding unit **2204** decodes the encoded image data received by the communication unit **2202**.

[0440] The decoding unit **2204** decodes the encoded image data by using said decoding methods described above.

[0441] The control unit **2206** controls other units in the smart phone **2200** in accordance with a user operation or commands received by the communication unit **2202**.

[0442] For example, the control unit **2206** controls a display unit **2208** so as to display an image decoded by the decoding unit **2204**.

[0443] While the present invention has been described with reference to embodiments, it is to be understood that the invention is not limited to the disclosed embodiments. It will be appreciated by those skilled in the art that various

changes and modification might be made without departing from the scope of the invention, as defined in the appended claims. All of the features disclosed in this specification (including any accompanying claims, abstract and drawings), and/or all of the steps of any method or process so disclosed, may be combined in any combination, except combinations where at least some of such features and/or steps are mutually exclusive. Each feature disclosed in this specification (including any accompanying claims, abstract and drawings) may be replaced by alternative features serving the same, equivalent or similar purpose, unless expressly stated otherwise. Thus, unless expressly stated otherwise, each feature disclosed is one example only of a generic series of equivalent or similar features.

[0444] It is also understood that any result of comparison, determination, assessment, selection, execution, performing, or consideration described above, for example a selection made during an encoding or filtering process, may be indicated in or determinable/inferable from data in a bitstream, for example a flag or data indicative of the result, so that the indicated or determined/inferred result can be used in the processing instead of actually performing the comparison, determination, assessment, selection, execution, performing, or consideration, for example during a decoding process.

[0445] In the claims, the word “comprising” does not exclude other elements or steps, and the indefinite article “a” or “an” does not exclude a plurality. The mere fact that different features are recited in mutually different dependent claims does not indicate that a combination of these features cannot be advantageously used.

[0446] Reference numerals appearing in the claims are by way of illustration only and shall have no limiting effect on the scope of the claims.

1. A method of decoding video data from a bitstream, the bitstream comprising (a) a picture parameter set, (b) a picture header syntax structure comprising syntax elements to be used when decoding one or more slices, and (c) a slice header comprising syntax elements to be used when decoding a slice, wherein the method comprises:

decoding syntax elements comprising (a) a first flag indicating whether the picture header syntax structure is present in the slice header, and (b) a second flag which is in the picture parameter set; and

decoding the video data from the bitstream using the decoded syntax elements,

wherein in a case where the second flag has a first value, the second flag indicates that Adaptive Loop Filter (ALF) information could be present in the picture header syntax structure and is not present in the slice header referring to the picture parameter set that does not contain the picture header syntax structure, and in a case where the second flag has a second value different from the first value, the second flag indicates that ALF information is not present in the picture header syntax structure and could be present in the slice header referring to the picture parameter set,

wherein in a case where the second flag has the first value, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header,

wherein the ALF information comprises an adaptation parameter set id for ALF, and an adaptation parameter set (APS) indicated by the adaptation parameter set id

includes a third flag which indicates, for luma, if clipping indexes related to ALF are to be decoded, and wherein in a case where Reference Picture List (RPL) information is present in the picture header syntax structure, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header.

2. The method according to claim 1, wherein the first flag is included in the slice header.

3. The method according to claim 1, wherein the third flag is an `alf_luma_clip_flag`, and each of the clipping indexes correspond to an `alf_luma_clip_idx`.

4. The method according to claim 1, wherein the APS indicated by the adaptation parameter set id comprises information corresponding to a number of filters for luma.

5. A method of encoding video data into a bitstream, the bitstream comprising (a) a picture parameter set, (b) a picture header syntax structure comprising syntax elements to be used when decoding one or more slices, and (c) a slice header comprising syntax elements to be used when decoding a slice, wherein the method comprises:

encoding the video data using syntax elements comprising

(a) a first flag indicating whether the picture header syntax structure is present in the slice header or not, and (b) a second flag, which is in the picture parameter set,

wherein in a case where the second flag has a first value, the second flag indicates that Adaptive Loop Filter (ALF) information could be present in the picture header syntax structure and is not present in the slice header referring to the picture parameter set that does not contain the picture header syntax structure, and in a case where the second flag has a second value different from the first value, the second flag indicates that ALF information is not present in the picture header syntax structure and could be present in the slice header referring to the picture parameter set,

wherein in a case where the second flag has the first value, the first flag shall have a value indicating that the picture header syntax structure is not present in the slice header,

wherein the ALF information comprises an adaptation parameter set id for ALF, and an adaptation parameter set (APS) indicated by the adaptation parameter set id includes a third flag which indicates, for luma, if clipping indexes related to ALF are to be decoded, and wherein in a case where Reference Picture List (RPL) information is present in the picture header syntax structure, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header.

6. The method according to claim 5, wherein the first flag is included in the slice header.

7. A device for decoding video data from a bitstream, the bitstream comprising (a) a picture parameter set, (b) a picture header syntax structure comprising syntax elements to be used when decoding one or more slices, and (c) a slice header comprising syntax elements to be used when decoding a slice, wherein the device is configured to:

decode syntax elements comprising (a) a first flag indicating whether the picture header syntax structure is present in the slice header or not, and (b) a second flag which is in the picture parameter set; and

decode the video data from the bitstream using the decoded syntax elements,

wherein in a case where the second flag has a first value, the second flag indicates that Adaptive Loop Filter (ALF) information could be present in the picture header syntax structure and is not present in the slice header referring to the picture parameter set that does not contain the picture header syntax structure, and in a case where the second flag has a second value different from the first value, the second flag indicates that ALF information is not present in the picture header syntax structure and could be present in the slice header referring to the picture parameter set,

wherein in a case where the second flag has the first value, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header,

wherein the ALF information comprises an adaptation parameter set id for ALF, and an adaptation parameter set (APS) indicated by the adaptation parameter set id includes a third flag which indicates, for luma, if clipping indexes related to ALF are to be decoded, and

wherein in a case where Reference Picture List (RPL) information is present in the picture header syntax structure, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header.

8. A device for encoding video data into a bitstream, the bitstream comprising (a) a picture parameter set, (b) a picture header syntax structure comprising syntax elements to be used when decoding one or more slices,

and (c) a slice header comprising syntax elements to be used when decoding a slice, wherein the device is configured to:

encode the video data using syntax elements comprising

(a) a first flag indicating whether the picture header syntax structure is present in the slice header or not, and

(b) a second flag, which is in the picture parameter set, wherein in a case where the second flag has a first value, the second flag indicates that Adaptive Loop Filter (ALF) information could be present in the picture header syntax structure and is not present in the slice header referring to the picture parameter set that does not contain the picture header syntax structure, and in a case where the second flag has a second value different from the first value, the second flag indicates that ALF information is not present in the picture header syntax structure and could be present in the slice header referring to the picture parameter set,

wherein in a case where the second flag has the first value, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header,

wherein the ALF information comprises an adaptation parameter set id for ALF, and an adaptation parameter set (APS) indicated by the adaptation parameter set id includes a third flag which indicates, for luma, if clipping indexes related to ALF are to be decoded, and

wherein in a case where Reference Picture List (RPL) information is present in the picture header syntax structure, the first flag shall have a value that indicates that the picture header syntax structure is not present in the slice header.

* * * * *